CLAUDE_CODE_WORKFLOW

Helios â€” Claude Code Operating Manual

`CLAUDE_CODE_WORKFLOW.md` Â· v1.0 Â· 2026-06-03 Â· *How to build Helios efficiently with Claude Code on Claude Pro, one developer, without hallucination.*

> This manual is the *how-to-drive-the-tool* layer. `IMPLEMENTATION_PLAYBOOK.md` says **what** to build and **why**; this says **how to get Claude Code to produce it correctly**. The decisive fact: **the code for every milestone already lives in the repo docs** â€” so Claude Code's job is to *wire, adapt, and test* it, not invent it. Hand it the source of truth and it cannot hallucinate; let it guess and it will. Build to the lean scope (`LEAN_SCOPE.md`): **[Wk1-MVP]** M0â€“M5 + the decomposition, **[Wk2-v1]** one MCP server + one grounded LLM brief + a small honest eval, **[v2-later]** the rest.

Before you start: configure Claude Code for this repo

1. **Move `CLAUDE.md` to the repo root.** It currently sits in `docs/` . At the root it **auto-loads into every Claude Code session**, pinning the canonical names, conventions, and grounding rules â€” this is your single most important hallucination firewall. *(Also fix the two known doc drifts so Claude reads truth: the registry is `semantic_layer.yaml` , not `semantic_models.yml` ; the benchmark has 50 scenarios, not 34.)*
2. **Register the MCP server** (from M6) in `.mcp.json` / `mcp_servers.yaml` so Claude Code can only reach data through governed tools.
3. **Create the project slash commands** (`.claude/commands/`): `/resume` , `/new-model` , `/new-metric` , `/new-mcp-tool` , `/review-sql` , `/new-scenario` . They encode the prompt anatomy + review rubric so you don't retype them.
4. **One milestone per session.** `/clear` (or a fresh session) between milestones â€” no context bleed, lower usage, less drift.

The operating doctrine (9 habits that prevent hallucination)

- **D1 â€” Root `CLAUDE.md` is law.** Canonical names auto-load; Claude can't drift on `session_key` , `reached_*` , the 10 channel groups, the grains.
- **D2 â€” Attach the source-of-truth section** for every file. Claude *copies/adapts*; it doesn't guess.
- **D3 â€” Attach the real upstream files** (the actual columns it must reference) so it physically can't invent columns.
- **D4 â€” Demand citations:** "cite the exact doc section you used for each block."
- **D5 â€” Forbid invention:** "use ONLY names in the attached registry / GA4 schema; if a name isn't there, STOP and ASK â€” don't guess."
- **D6 â€” Let the machine catch it:** `dbt parse` / `compile` and BigQuery `dry_run` fail on non-existent columns; tests fail on wrong logic. Compile + dry-run + test *before* accepting.
- **D7 â€” One name source:** `semantic_layer.yaml` + the GA4 schema are the only places metric/column names come from â€” keep them attached.
- **D8 â€” `/clear` between milestones.**
- **D9 â€” Plan mode first** for any multi-file milestone â€” review the plan against the spec before a line of code exists.

The prompt anatomy (every codegen prompt)

```
[ROLE]        Implement Helios milestone M<n>, file <path>.
[SOURCE]      Copy/adapt the code in @<DOC Â§X>. Follow CLAUDE.md (auto-loaded).
[UPSTREAM]    Reference columns/refs ONLY from these: @<upstream files>.
[TASK]        <one concrete file/behavior>.
[CONSTRAINTS] Use ONLY canonical names from the semantic layer / GA4 schema.
              Do NOT invent columns, metrics, or tables. If a name isn't in the
              attached sources, STOP and ASK.
[TEST-FIRST]  Write the test (<test>) first, then the code, then run <command>
              and show me the output.
[OUTPUT]      Cite the doc section used for each block.
```

The review rubric (never accept code that fails any of R1â€“R6)

	Check
R1	It **cites** the spec section it used.
R2	It **parses/compiles** (`dbt parse` / `compile` , python import) â€" no ref errors.
R3	Claude **ran the milestone's test** and showed it pass (don't take its word â€" see the output).
R4	Every column/metric name is **canonical** (in the registry / GA4 schema).
R5	The **diff matches conventions** (naming, materialization, `session_key` , `reached_*` , money, grains) â€" *you read it*.
R6	For SQL: `dry_run` **clean** (catches hallucinated columns + cost).

Claude Pro budget rules

- Plan mode is cheap; bad regenerated code is expensive â€" **plan before coding**.
- `/clear` between milestones (smaller context = less usage + less drift).
- Batch one milestone's related files in **one focused session**.
- **Don't burn budget running the product's LLM brief/agents repeatedly during dev.** Build and test the *deterministic* layers (dbt, stats, the MCP server) with Claude Code; spend the LLM budget only when validating the M7/v1 brief.
- Subagents multiply usage â€" reserve them (e.g., a single code-review subagent), don't use them for everything.

Session opener (copy-paste)

```
We're building Helios milestone M<n> (<name>). CLAUDE.md is loaded.
I'm attaching @<source-doc Â§X> as the source of truth and @<upstream files>.
PLAN FIRST â€" don't write code yet: list the files, the tests to write first, and
confirm you'll use only canonical names from the attached registry/GA4 schema.
Then wait for my go.
```

How to read the per-milestone sections

Each milestone below has five parts â€" **Context to load Â· Files to provide Â· Prompts to use Â· Preventing hallucinations Â· Reviewing generated code** â€" and a lean tag: **[Wk1-MVP]**, **[Wk2-v1]**, or **[v2-later]**. Build the tagged-for-now milestones; the `[v2-later]` ones include how you'd drive Claude Code to build them, but they're out of the 2-week / Pro budget.

M0 - Foundation, toolchain & dbt config [Wk1-MVP]

The very first move is **not** writing code. It is making Claude Code's memory correct. `CLAUDE.md` currently sits in `docs/` , so it does **not** auto-load at the start of a session and Claude has no canonical-name anchor (D1 violated by default). M0's first action moves it to the repo root, then scaffolds the dbt project skeleton by *copying* the configs that already exist verbatim in `DBT_GUIDE.md` sec 1 - Claude wires, it does not design.

Context to load

- Open `DBT_GUIDE.md` **sec 1** (the project tree + `dbt_project.yml` / `packages.yml` / `profiles.yml` + Conventions) - the source of truth for every config file.
- Open `MCP_ARCHITECTURE.md` **sec 3** (the `mcp_servers.yaml` block) only for the stub.
- Confirm `CLAUDE.md` will auto-load: after the move, it must be at `helios/CLAUDE.md` (repo root). Until then, **attach it manually** in the M0 session so the canonical vocabulary (session_key, reached_*, 10 channel groups, layer prefixes) is in context.
- Reality check before scaffolding: the repo today contains only `docs/` , `models/semantic/` (the M5 `semantic_layer.yaml` already exists), `eval/` , and `PROJECT_STRUCTURE.md` . No `dbt_project.yml` yet. Do not let Claude assume a dbt project is already initialised.

Files to provide

- `@docs/architecture/DBT_GUIDE.md` (sec 1) - the literal text of `dbt_project.yml` , `packages.yml` , `profiles.yml` .
- `@docs/CLAUDE.md` (the file being moved) - so Claude knows the conventions and the target layout in sec 6.
- `@docs/architecture/MCP_ARCHITECTURE.md` (sec 3) - for the `mcp_servers.yaml` stub.
- Nothing else. M0 touches no SQL and no GA4 columns, so do not attach the data-model docs.

Prompts to use

First, the CLAUDE.md move (a one-liner, but do it as an explicit step so the rest of M0 benefits):

```
[ROLE] You are setting up the Helios repo for milestone M0. CLAUDE.md is the auto-load
memory file but it currently lives in docs/, so it does NOT auto-load.
[TASK] Move docs/CLAUDE.md to the repo root (helios/CLAUDE.md). Update the one self-
reference line in it ("CLAUDE.md  â†’ this file" already shows it at root, so the file body
is correct - only the path changes). Then update docs/planning/DEPENDENCY_MAP.md and any doc that
links to docs/CLAUDE.md to point at ./CLAUDE.md.
[CONSTRAINTS] Do not edit the body of CLAUDE.md other than fixing stale relative links.
Show me `git mv` and the diff. Do not create a copy - move it.
```

Then enter **plan mode** for the scaffold (Shift+Tab) and use:

```
[ROLE] You are implementing Helios milestone M0 (repo scaffold + dbt config). Follow
CLAUDE.md conventions (now auto-loaded at repo root).
[SOURCE OF TRUTH] Copy/adapt the four config files VERBATIM from DBT_GUIDE.md sec 1
(@docs/architecture/DBT_GUIDE.md): dbt_project.yml, packages.yml, profiles.yml. Add a .gitignore,
requirements.txt, and the mcp_servers.yaml STUB from MCP_ARCHITECTURE.md sec 3
(@docs/architecture/MCP_ARCHITECTURE.md) at the repo root.
[TASK] In plan mode, propose:
  1. dbt_project.yml at repo root - copy sec 1 exactly: vars (ga4_start_date 2020-11-01,
     ga4_end_date 2021-01-31, incremental_lookback_days 3, engagement_time_threshold_msec
     10000), the per-layer +configs (staging=view+access:private, intermediate=ephemeral,
     marts core=incremental insert_overwrite partitioned by event_date clustered by
     device_category,channel_group, dims=table). Set profile: helios.
  2. packages.yml - dbt_utils, dbt_expectations, dbt_project_evaluator, codegen with the
     exact version pins from sec 1.
  3. profiles.yml - helios profile, dev target oauth/helios_dev/location US/
     maximum_bytes_billed 5368709120; prod target as in sec 1.
  4. requirements.txt - dbt-bigquery, plus the Python deps the project will need (do NOT
     invent versions; pin dbt-bigquery≥1.7).
  5. .gitignore - target/, dbt_packages/, logs/, *.json keyfiles, .env, __pycache__/.
  6. mcp_servers.yaml STUB at repo root - copy the servers block from MCP_ARCHITECTURE.md
     sec 3; registry must point at ./models/semantic/semantic_layer.yaml.
  7. Create the empty model folder skeleton from CLAUDE.md sec 6 (models/staging,
     models/intermediate, models/marts/{core,finance,growth}, macros/, seeds/, tests/).
[CONSTRAINTS] Use ONLY the values present in the attached doc sections. Do NOT invent a
GCP project name other than helios-analytics (the value in sec 1). If a value is missing,
STOP and ASK - do not guess credentials, dataset names, or versions.
[OUTPUT] Show me the plan first. For each file, cite the DBT_GUIDE.md sec 1 sub-block it
came from. After I approve, write the files, then run `dbt deps` and `dbt debug` and show
the full output.
```

Preventing hallucinations

- D1 in action: the **whole point of M0** is to fix auto-load. Verify `CLAUDE.md` is at the root with `git ls-files CLAUDE.md` (must return `CLAUDE.md`, not `docs/CLAUDE.md`); a fresh session afterward should show it loaded.
- D2/D5: the configs are *transcribed*, not authored. The biggest hallucination risk is Claude "improving" values - inventing a `maximum_bytes_billed`, a different `location`, a `priority`, or a 6th MCP server. Pin them: `maximum_bytes_billed: 5368709120` (5 GiB), `location: US`, `project: helios-analytics`, exactly the 5 servers from MCP sec 3. Grep the result: `Select-String "5368709120|location: US|helios-analytics" dbt_project.yml,profiles.yml`.
- The mcp_servers.yaml is a **stub** - Claude must not flesh out tool implementations in M0 (those are M6). The registry path must be `./models/semantic/semantic_layer.yaml` and that file already exists, so a typo'd path is catchable.
- D6 - let the machine catch it: `dbt debug` is the M0 acceptance gate. It validates `profiles.yml` parses, the BigQuery connection authenticates via ADC, `dbt_project.yml` is well-formed, and packages resolve. If Claude hallucinated a malformed `+config` key, `dbt parse` / `dbt debug` fails immediately.

Reviewing generated code

- R2/R3: run and show `dbt deps` (packages install clean) then `dbt debug` - **every check must be green**: profiles.yml found, dbt_project.yml valid, connection ok. Do not accept M0 without a green `dbt debug`.
- R5 (read the diff against conventions): confirm staging/intermediate are `+access: private`; marts core is `incremental` / `insert_overwrite` /partition `event_date` /cluster `device_category, channel_group` / `require_partition_filter: true`; the four dims override back to `table` + `require_partition_filter: false`. These are the load-bearing cost/governance controls from sec 1.
- Confirm `vars` are present and exact (the build window and the 10000 ms engagement threshold) - downstream macros read `var('engagement_time_threshold_msec')`, so a missing var breaks M1/M3.
- Confirm `.gitignore` excludes `target/`, `dbt_packages/`, and any `*.json` keyfile (no service-account key should ever be committed - dev uses OAuth/ADC per sec 1).

- Confirm `CLAUDE.md` moved (not copied) and no dangling `docs/CLAUDE.md` link remains.
- /clear before M1.

M1 - Macros, sources & seed [Wk1-MVP]

M1 builds the shared abstractions that every downstream model depends on: the **four macros** (`get_event_param` , `sessionize` , `channel_group` / `channel_group_case` , and the custom generic test `test_revenue_reconciles`), the `src_ga4` source declaration, and the `channel_group_mapping.csv` seed. All of this exists verbatim in `DBT_GUIDE.md` sec 3.4 (macros), sec 2 / sec 3.1 (source), and sec 6.4 (the test). Claude copies the macro bodies exactly - the `channel_group_case()` regex precedence and the `sessionize()` MD5 concat order are canonical and must not be paraphrased.

Context to load

- `DBT_GUIDE.md` **sec 3.4** - the three macro bodies (`get_event_param` , `sessionize` , `channel_group_case`) verbatim.
- `DBT_GUIDE.md` **sec 2 + sec 3.1** - the `src_ga4` source declaration (`_ga4__sources.yml` / `src_ga4.yml`): `database: bigquery-public-data` , `schema: ga4_obfuscated_sample_ecommerce` , `identifier: 'events_*'` , the freshness contract (production-real, sample-informational).
- `DBT_GUIDE.md` **sec 6.4** - the `test_revenue_reconciles` custom generic test body.
- `CLAUDE.md` (auto-loaded) sec 4-5 - the 10 channel groups, the canonical `session_key` expression, the `traffic_source` gotcha, money/ `_in_usd` rule.

Files to provide

- `@docs/architecture/DBT_GUIDE.md` (secs 2, 3.1, 3.4, 6.4) - the literal macro/source/test text.
- The **GA4 source schema** so the source declaration references real raw columns: attach the column list from `DBT_GUIDE.md` sec 2's `_ga4__sources.yml` (event_date, event_timestamp, event_name, user_pseudo_id, ecommerce.transaction_id) - and note the GA4 export schema is public (`bigquery-public-data.ga4_obfuscated_sample_ecommerce`), so the macros may only reference fields that exist there (`event_params[].key` , `event_params[].value.{string,int,float,double}_value` , `traffic_source.{source,medium,name}` , `device.*` , `geo.*` , `ecommerce.*`).
- `@models/semantic/semantic_layer.yaml` - the registry exists already; `channel_group` is a canonical dimension there. Keep it open so the seed's `channel_group` values match the 10 governed names exactly.

Prompts to use

Set up the slash commands first (cheap, paid off all week). Create `.claude/commands/new-model.md` and `.claude/commands/new-metric.md` :

```
[TASK] Create two custom slash commands for this repo.
1. .claude/commands/new-model.md - "/new-model <layer> <name>": scaffold a dbt model file
   at models/<layer>/<name>.sql + a stub entry in that folder's _*__models.yml, using the
   conventions in CLAUDE.md (layer prefix stg_/int_/fct_/dim_, materialization per
   dbt_project.yml, snake_case). The command body must instruct: attach the relevant
   DBT_GUIDE.md section, reference only real upstream columns, write the schema test
   (unique+not_null on the grain key) BEFORE the SQL, then run `dbt parse`.
2. .claude/commands/new-metric.md - "/new-metric <name>": add a metric to
   models/semantic/semantic_layer.yaml (NOT hand-written SQL), add its test, and re-run
   validate_semantic.py. The body must forbid inventing column names not in the registry.
Use $ARGUMENTS in each command. Show me both files.
```

The macros (one focused session, plan mode):

```
[ROLE] You are implementing Helios M1, files macros/get_event_param.sql,
macros/sessionize.sql, macros/channel_group.sql, and tests/generic/
test_revenue_reconciles.sql. Follow CLAUDE.md conventions (auto-loaded).
[SOURCE OF TRUTH] Copy the macro bodies VERBATIM from DBT_GUIDE.md sec 3.4
(@docs/architecture/DBT_GUIDE.md) and the test from sec 6.4. Do not rewrite or "clean up" the regexes
or the MD5 concat.
[UPSTREAM] These macros run against the raw GA4 export. Reference ONLY columns that exist
in bigquery-public-data.ga4_obfuscated_sample_ecommerce: event_params (ARRAY<STRUCT key,
value>), traffic_source.{source,medium,name}, device.*, geo.*, ecommerce.* per the
attached sec 2 column list.
[TASK]
  - get_event_param(key, type='string'): the typed correlated-subquery extractor (string/
    int/float/double value slots) exactly as sec 3.4.
  - sessionize(): TO_HEX(MD5(CONCAT(user_pseudo_id,'-',CAST(ga_session_id AS STRING)))) -
    the ONE canonical session_key expression. Never FARM_FINGERPRINT.
  - channel_group()/channel_group_case(): the 10-group CASE (Direct, Organic Search, Paid
```

```
    Search, Display, Paid Social, Organic Social, Email, Affiliates, Referral, Other),
    gclid-aware, in the exact precedence from sec 3.4. NO 11th group, no "Paid Other".
  - test_revenue_reconciles: the custom generic test from sec 6.4 (0.5% tolerance vs raw
    purchase_revenue_in_usd where event_name='purchase').
[CONSTRAINTS] Use ONLY canonical names from CLAUDE.md / semantic_layer.yaml. If you need a
column not in the attached GA4 schema, STOP and ASK.
[TEST-FIRST] Then run `dbt parse` (must compile the macros with no Jinja/ref errors).
[OUTPUT] Cite the sec 3.4 / 6.4 sub-block for each macro.
```

The source + seed:

```
[ROLE] Helios M1, files models/staging/_ga4__sources.yml and
seeds/channel_group_mapping.csv. Follow CLAUDE.md conventions.
[SOURCE OF TRUTH] Copy the src_ga4 declaration from DBT_GUIDE.md sec 2/3.1
(@docs/architecture/DBT_GUIDE.md): name src_ga4, database bigquery-public-data, schema
ga4_obfuscated_sample_ecommerce, table events identifier 'events_*', the freshness
block + loaded_at_field, and the not_null source-column tests.
[TASK] Write _ga4__sources.yml exactly as the doc. Then create the seed
channel_group_mapping.csv (columns: source, medium, channel_group) whose channel_group
values are ONLY the 10 canonical groups - cross-check against channel_group_case() and
semantic_layer.yaml (@models/semantic/semantic_layer.yaml).
[CONSTRAINTS] The 10 channel_group values must match the macro exactly; no 11th value.
[TEST-FIRST] Run `dbt parse` then `dbt seed --select channel_group_mapping` and show output.
[OUTPUT] Cite sec 2/3.1.
```

Preventing hallucinations

- D3/D5 (real upstream columns): the single biggest M1 risk is the macros referencing a GA4 field that doesn't exist or has the wrong path. Pin: `event_params` is `ARRAY<STRUCT<key STRING, value STRUCT<string_value, int_value, float_value, double_value, … >>>` ; session id lives in `event_params.ga_session_id` (an int param), **not** a top-level column; source/medium are session-scoped `event_params.source/medium` , with `traffic_source.*` as first-touch fallback only (the gotcha in CLAUDE.md sec 5). If Claude reaches for `traffic_source` as the primary source, that is the canonical bug - reject it.
- D4 (cite): require Claude to cite sec 3.4 for each macro. The `channel_group_case()` regex precedence (Direct -> Paid Search -> Paid Social -> Display -> Organic Search -> ...) is order-sensitive; a reordered CASE produces silently wrong attribution. Diff the regexes against sec 3.4 character-for-character.
- D5 (no invention): the seed's `channel_group` column must contain exactly the 10 canonical strings. Run `Get-Content seeds/channel_group_mapping.csv | Select-Object -Skip 1 | ForEach-Object { ($_ -split ',')[2] } | Sort-Object -Unique` and confirm the set is the 10 names with no extras (no "Paid Other", no 11th).
- D6 (machine catch): `dbt parse` compiles the macros and the source YAML; a Jinja typo or a malformed `source()` declaration fails here before any model uses it. There are no models yet, so `dbt parse` is the cheap, fast M1 gate.

Reviewing generated code

- R2: `dbt parse` clean (macros compile, source YAML valid). `dbt seed --select channel_group_mapping` loads without a type error (the seed `+column_types` from M0's dbt_project.yml force source/medium/channel_group to string).
- R4 (canonical names): grep the macro file - `sessionize` must contain `to_hex(md5(concat(` and `user_pseudo_id` / `ga_session_id` , and must **not** contain `farm_fingerprint` . `channel_group_case` must contain all 10 group literals and no others.
- R5: confirm `get_event_param` selects the right value slot per type (string->string_value, int->int_value, etc.) and uses `where ep.key = '{{ key }}'` with `limit 1` - a wrong slot silently returns null for every int param (ga_session_id, engagement_time_msec), which would zero out sessionization downstream.
- R1: each macro block cites its sec 3.4 origin; the test cites sec 6.4.
- The `test_revenue_reconciles` body: confirm it filters `event_name = 'purchase'` on the raw side and uses the 0.5% relative-drift tolerance - it can't be exercised until a revenue mart exists (M4), but it must `dbt parse` now.
- /clear before M2.

M2 - Staging models [Wk1-MVP]

M2 builds the two staging views - `stg_ga4__events.sql` (1:1 typed/renamed events, `session_key` via `sessionize()`) and `stg_ga4__event_params.sql` (the long unnested param table) - plus `stg_ga4__schema.yml` . Both are verbatim in `DBT_GUIDE.md` sec 3.2, 3.3, 3.5. Staging is *pure projection*: **no joins, no aggregations, no `SELECT *` against source, no business logic** (sec 3 preamble). This is the first milestone that produces real data, so it's the first chance for hallucinated GA4 columns to leak - the source schema must be attached.

Context to load

- `DBT_GUIDE.md` **sec 3 preamble** (the staging discipline rules) + **sec 3.2** (`stg_ga4__events`) + **sec 3.3** (`stg_ga4__event_params`) + **sec 3.5** (`stg_ga4__schema.yml`).
- The **M1 macros** - staging *calls* `get_event_param()` and `sessionize()` . Keep `macros/get_event_param.sql` and `macros/sessionize.sql` open so Claude uses the real macro signatures, not hand-written UNNEST.
- The GA4 source schema (the `_ga4__sources.yml` column list from M1) - the authoritative list of raw columns staging may project.
- `CLAUDE.md` sec 5 - layer prefix `stg_ga4__` , materialization `view` , money `_in_usd` only, the `traffic_source` first-touch gotcha.

Files to provide

- `@docs/architecture/DBT_GUIDE.md` (sec 3, esp. 3.2/3.3/3.5) - the literal staging SQL + schema.yml.
- `@macros/get_event_param.sql` and `@macros/sessionize.sql` - the real macros M2 must call.
- `@models/staging/_ga4__sources.yml` - the `source('src_ga4','events')` target and the GA4 column contract.
- Use `/new-model staging stg_ga4__events` (the command built in M1) to scaffold, then fill from sec 3.2.

Prompts to use

Plan mode first:

```
[ROLE] You are implementing Helios M2, files models/staging/stg_ga4__events.sql,
models/staging/stg_ga4__event_params.sql, models/staging/stg_ga4__schema.yml. Follow
CLAUDE.md conventions (auto-loaded).
[SOURCE OF TRUTH] Copy/adapt the SQL VERBATIM from DBT_GUIDE.md sec 3.2, 3.3, 3.5
(@docs/architecture/DBT_GUIDE.md). These are pure projection views.
[UPSTREAM] Reference ONLY: the source via {{ source('src_ga4','events') }}
(@models/staging/_ga4__sources.yml) and the macros get_event_param / sessionize
(@macros/get_event_param.sql, @macros/sessionize.sql). Use ONLY raw GA4 columns that
exist in the attached source schema.
[TASK]
  - stg_ga4__events: parse_date('%Y%m%d', _table_suffix) AS event_date; explicit column
    list (NO SELECT *); ga_session_id / ga_session_number / page_location / session_engaged
    / engagement_time_msec / source / medium / campaign / gclid via get_event_param() with
    the correct types; traffic_source.{source,medium,name} as first_touch_* FALLBACK;
    device.* and geo.* light-flattened; ecommerce.*_in_usd money columns; session_key via
    sessionize() in the final CTE. Materialized: view.
  - stg_ga4__event_params: UNNEST(event_params) into the long (event x param key) table
    with string/int/float/double value slots + a coalesced value_string. This is the ONLY
    place UNNEST(event_params) appears. Materialized: view.
  - stg_ga4__schema.yml: the column docs + tests from sec 3.5 (session_key not_null where
    ga_session_id is not null; event_date/event_timestamp/user_pseudo_id not_null;
    param_key not_null; purchase_revenue_in_usd accepted_range min 0).
[CONSTRAINTS] NO joins, NO aggregations, NO SELECT * against the source, NO business logic
(that's intermediate/M3). Use ONLY canonical names. If a GA4 column you need is not in the
attached source schema, STOP and ASK.
[TEST-FIRST] Write/confirm stg_ga4__schema.yml tests, then run `dbt parse`, then
`dbt build --select staging` and show me the output (models built + tests passed).
[OUTPUT] Cite sec 3.2 / 3.3 / 3.5 per block.
```

If `dbt build --select staging` surfaces a real-data mismatch (e.g. a struct path differs on the sample), use:

```
The dbt build --select staging output shows <paste the exact BigQuery error>. Do NOT
guess a fix. Run `dbt show --select stg_ga4__events --limit 5` and
`{{ source('src_ga4','events') }}` schema via a one-off describe to find the real column
path, cite it, then correct ONLY that projection. Re-run `dbt build --select staging`.
```

Preventing hallucinations

- D3 (real columns) is the headline tactic for M2: staging is the layer that touches raw GA4, so a hallucinated column path is the prime failure. Pin the known-tricky paths: session id is `event_params.ga_session_id` via `get_event_param('ga_session_id','int')` (**not** a top-level `ga_session_id`); browser is `device.web_info.browser` ; money is `ecommerce.purchase_revenue_in_usd` / `refund_value_in_usd` / `shipping_value_in_usd` / `tax_value_in_usd` (USD twins only - never the non-USD `purchase_revenue`).
- D6 - the machine is the catcher here, and it is decisive: `dbt build --select staging` actually runs the view against BigQuery. A hallucinated column (e.g. `device.browser` instead of `device.web_info.browser` , or a non-existent `ecommerce.gross_revenue`) fails with "Field not found" - you cannot fake-pass this. This is the staging-specific D6: compile is not enough, the build must touch the warehouse.
- D2/D5: enforce the "no business logic in staging" rule - if Claude adds a `WHERE event_name IN ( … )` filter, a `GROUP BY` , or a join to the seed, reject it; that logic belongs to M3 intermediate. The only `WHERE` allowed is the `_TABLE_SUFFIX` / `var('ga4_start_date')` shard prune.
- Confirm `session_key` comes from `sessionize()` (the macro), not an inline `MD5( … )` - a hand-inlined key is a D5 violation even if it's correct, because it duplicates the single source of truth.

Reviewing generated code

- R3: `dbt build --select staging` runs both views and their schema.yml tests - **show the pass**. This is the M2 acceptance gate.
- R2: no "Field not found" / "Unrecognized name" errors in the build log (proves no hallucinated columns - the staging-layer R6 equivalent, since views are dry-run-cheap and the build itself is the schema validation).
- Key uniqueness / not_null (the milestone-specific checks): `stg_ga4__events.session_key` not_null where `ga_session_id is not null` passes; `event_date`, `event_timestamp`, `user_pseudo_id` not_null pass; `stg_ga4__event_params.param_key` not_null passes. session_key is **not** unique at staging grain (one row per event, many events per session) - do not add a `unique` test here; uniqueness is asserted at the session grain in M3/M4.
- R5 (read the diff vs conventions): both models are `materialized: view`; explicit column lists (grep for `select *` against `{{ source(` - there must be none; the only `select *` is over the already-explicit CTE in `stg_ga4__events`'s `final`); `UNNEST(event_params)` appears in exactly one file (`stg_ga4__event_params`); `_in_usd` money columns only.
- R4: every projected column maps to a real GA4 field or a `get_event_param()` call; no invented metric/column names.
- R1: each SQL block cites sec 3.2 / 3.3; schema.yml cites sec 3.5. Then /clear before M3 (the sessionization keystone gets its own clean session and golden tests first).

M3 â€" Sessionization & Funnel KEYSTONE [Wk1-MVP]

This is the highest-stakes milestone in the build. `int_ga4__sessionized` and `int_ga4__funnel_steps` reconstruct the two entities GA4 never ships as rows â€" the **session** and the **funnel** â€" and they **fail silently**: a wrong `MD5` concat order still yields a valid-looking key, a non-monotonic flag still returns a number, a `traffic_source` first-touch leak still returns rows. Nothing throws; the numbers are just quietly wrong, and every mart, metric, and agent diagnosis inherits the corruption. The non-negotiable operating rule for M3 is therefore **TEST-FIRST**: the golden-value unit tests + the two singular tests are written and watched fail *before* the model exists. The code already lives in the docs â€" Claude Code copies/adapts it, it does not invent it.

Context to load

- **Root** `CLAUDE.md` (auto-loads). It pins the four canonical names you will check by eye in every diff: `session_key = TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))`; `reached_*` max-downstream monotonic (`did_*` retired); `engaged_session = session_engaged = '1' OR engagement_time_msec ≥ 10000`; the `traffic_source` first-touch FALLBACK.
- **`@docs/architecture/DBT_GUIDE.md` Â§4** â€" the full copy-usable SQL for both keystones (Â§4.1 `int_ga4__sessionized`, Â§4.2 `int_ga4__funnel_steps`), the intermediate `schema.yml` (Â§4.3), and the keystone test budget (Â§4.4).
- **`@docs/architecture/DBT_GUIDE.md` Â§6.3 + Â§6.5** â€" the singular tests (`assert_funnel_monotonicity`, `assert_session_conversion_rate_bounds`) and the three golden-value `unit_tests` (`test_sessionize_key_construction`, `test_reached_flags_are_max_downstream`, `test_decomposition_inputs_additive`).
- **`@docs/architecture/DATA_MODEL.md` Â§5** â€" the session model as spec: Â§5.1 `session_key` + cookie-grain honesty, Â§5.3 the `traffic_source` FALLBACK, Â§5.4 engaged/new-user, Â§5.5 `int_ga4__sessionized` grain/PK/columns, Â§5.6 the `reached_*` monotonic chain.
- The **upstream models** Claude must reference for real columns: `@models/staging/stg_ga4__events.sql` (it provides `session_key`, `event_source` / `event_medium`, `first_touch_source` / `first_touch_medium`, `gclid`, `session_engaged`, `engagement_time_msec`, `ga_session_number`, `transaction_id`, `purchase_revenue_in_usd`) and `@macros/sessionize.sql` + `@macros/channel_group.sql`.

Run M3 in a **dedicated session after** `/clear`. Plan mode first (D9): ask for the file list + the test-first order, check it against Â§4 before any code is written.

Files to provide

Attach these and nothing else (a tight context is the anti-drift lever and the Pro-budget lever):

Attach	Why
`@docs/architecture/DBT_GUIDE.md` Â§4.1, Â§4.2, Â§4.3	source-of-truth SQL + intermediate schema tests (D2)
`@docs/architecture/DBT_GUIDE.md` Â§6.3, Â§6.5	the singular + golden-value tests to write FIRST
`@docs/architecture/DATA_MODEL.md` Â§5.1, Â§5.3, Â§5.5, Â§5.6	grain/PK/column spec + the FALLBACK + the monotonic chain
`@models/staging/stg_ga4__events.sql`	the real upstream columns (D3 â€" so it can't invent a column name)
`@macros/sessionize.sql`, `@macros/channel_group.sql`	the single sources of truth it must call, not re-derive

Prompts to use

Prompt 1 â€" write the golden tests FIRST (this is the gate, not the model):

```
[ROLE] You are implementing Helios milestone M3. Write ONLY the keystone tests, before any model SQL.
[SOURCE OF TRUTH] Copy/adapt the three unit_tests in @docs/architecture/DBT_GUIDE.md Â§6.5 into
  models/intermediate/_int__unit_tests.yml, and the two singular tests in Â§6.3 into
  tests/assert_funnel_monotonicity.sql and tests/assert_session_conversion_rate_bounds.sql.
  Follow CLAUDE.md conventions (auto-loaded).
[TASK] Reproduce: test_sessionize_key_construction (u1+100 collapses 2 events to 1 session,
  channel_group='Paid Search' from gclid+cpc, engaged), test_reached_flags_are_max_downstream
  (a purchase-only session sets EVERY reached_* true; an add_to_cart-only session sets
  reached_view_item+reached_add_to_cart true and downstream false), and the monotonicity
  singular SELECT (returns offending sessions where reached_purchase > reached_add_payment_info > ...).
[CONSTRAINTS] Use ONLY canonical names: session_key, reached_view_item/add_to_cart/begin_checkout/
  add_shipping_info/add_payment_info/purchase, session_revenue, channel_group. Do NOT use did_*.
  Do NOT invent a column. If a name is not in Â§6.5/Â§6.3 or stg_ga4__events, STOP and ASK.
[TEST-FIRST] Then run `dbt parse` to confirm the YAML + SQL parse. Do NOT write the models yet.
[OUTPUT] Cite the exact doc subsection (Â§6.5 / Â§6.3) used for each test.
```

Prompt 2 â€” implement `int_ga4__sessionized` to pass the sessionization unit test:

```
[ROLE] You are implementing Helios milestone M3, file models/intermediate/int_ga4__sessionized.sql.
[SOURCE OF TRUTH] Copy/adapt the SQL in @docs/architecture/DBT_GUIDE.md Â§4.1; cross-check grain/PK/columns
  against @docs/architecture/DATA_MODEL.md Â§5.5 and the FALLBACK rule in Â§5.3. Follow CLAUDE.md (auto-loaded).
[UPSTREAM] Reference only @models/staging/stg_ga4__events.sql for columns and call
  {{ sessionize() }} (@macros/sessionize.sql) and {{ channel_group_case( ... ) }} (@macros/channel_group.sql).
[TASK] One row per session (group by user_pseudo_id, ga_session_id; drop null ga_session_id).
  Derive: landing_page = page_location of the EARLIEST event
  (array_agg( ... order by event_timestamp limit 1)); session_source/medium = COALESCE(session-scoped
  event_source/medium, traffic_source first_touch_*) â€” the FALLBACK; channel_group via
  channel_group_case() on the RESOLVED source/medium + gclid; engaged_session = session_engaged='1'
  OR engagement_time_msec ≥ 10000; is_new_user = (ga_session_number = 1). Materialize ephemeral.
[CONSTRAINTS] session_key comes ONLY from sessionize(); channel_group ONLY from channel_group_case()
  on the resolved (post-fallback) source/medium â€” NEVER from the raw traffic_source struct.
  Use ONLY columns present in stg_ga4__events. If one is missing, STOP and ASK â€” do not guess.
[TEST-FIRST] Make test_sessionize_key_construction pass: run
  `dbt build --select int_ga4__sessionized` and show me the test output.
[OUTPUT] Cite the Â§4.1 / Â§5.3 / Â§5.5 line behind each derived column.
```

Prompt 3 â€” implement `int_ga4__funnel_steps` to pass the monotonicity tests:

```
[ROLE] You are implementing Helios milestone M3, file models/intermediate/int_ga4__funnel_steps.sql.
[SOURCE OF TRUTH] Copy/adapt @docs/architecture/DBT_GUIDE.md Â§4.2; cross-check the monotonic chain in
  @docs/architecture/DATA_MODEL.md Â§5.6. Follow CLAUDE.md (auto-loaded).
[UPSTREAM] Reference only @models/staging/stg_ga4__events.sql.
[TASK] One row per session. Each reached_X = LOGICAL_OR(event_name IN {X and ALL downstream stages})
  (max-downstream). session_revenue = deduped purchase revenue: one value per (session_key,
  transaction_id) via MAX over the partition, then SUM across distinct transactions, COALESCE 0.
  Materialize ephemeral.
[CONSTRAINTS] reached_* names are canonical and max-downstream â€” did_* is FORBIDDEN. The stage sets
  must nest exactly: reached_view_item's IN-list contains all six stages, reached_purchase's only
  'purchase'. Do NOT invent stage names. If unsure, STOP and ASK.
[TEST-FIRST] Make test_reached_flags_are_max_downstream pass first:
  `dbt build --select int_ga4__funnel_steps`. Then run
  `dbt build --select int_ga4__funnel_steps,test_type:unit` and show output.
[OUTPUT] Cite Â§4.2 / Â§5.6 for the flag logic and the dedup logic.
```

Preventing hallucinations

- **Golden tests are the firewall (D6).** The two keystones cannot be caught by `not_null` / `unique` alone â€” those test shape, not correctness. The `test_reached_flags_are_max_downstream` unit test runs on synthetic input every build in <1s and is what catches a non-monotonic flag *before* any real data flows; demand it green before accepting the model. `dbt build` (not `run` then `test`) interleaves test execution in DAG order so a failed keystone test aborts before the poison reaches `fct_funnel`.
- **Pin the four exact expressions in the prompt and re-verify each by eye.** The `session_key` concat order (`user_pseudo_id` then `'-'` then `CAST(ga_session_id AS STRING)`), the FALLBACK direction (`COALESCE(event_source, first_touch_source)` â€” session first, first-touch second, never the reverse), the `reached_*` IN-list nesting, and `engaged_session`'s `≥` (not `>`). A reversed FALLBACK is a Simpson's-paradox-grade silent bug.
- **Forbid `did_*` explicitly (D5).** `grep -rn "did_" models/intermediate/` must return nothing except the deliberately-retained `did_session_start` denominator anchor if Â§4.2 uses it; any other `did_` flag means Claude reintroduced the retired naming.
- **`channel_group` only via the macro, on the resolved source/medium (D7).** If Claude inlines a `CASE WHEN ... THEN 'Paid Search'`, reject it â€” channel logic lives only in `channel_group_case()`; an 11th group or a hand-rolled CASE is a hard error.
- **Attach `stg_ga4__events.sql` so column names are physical (D3).** `event_source` / `event_medium` (session-scoped) vs `first_touch_source` / `first_touch_medium` (user first-touch) are easy to swap â€” having the real file open means Claude reads the names

rather than guessing.

Reviewing generated code

Apply the rubric, specialized to the keystones:

- **R3 (ran the test) â€" the decisive check.** Did Claude show `test_sessionize_key_construction` and `test_reached_flags_are_max_downstream` *passing*? No green unit test â‡' do not accept. Then `dbt test --select int_ga4__sessionized int_ga4__funnel_steps`.
- **R5 (read the diff by eye)** â€" read these three lines against the spec, every time: (1) the `session_key` expression matches `TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))` character-for-character (it should come from `sessionize()`, so confirm the call, not a re-typed expression); (2) the FALLBACK is `COALESCE(<session-scoped>, <first-touch>)` and not reversed; (3) the `reached_*` roll-forward â€" `reached_view_item` LOGICAL_OR's the set of all six downstream stages, each subsequent flag drops the leftmost stage, `reached_purchase` is `'purchase'` only.
- **R2/R6 (compile + dry_run)** â€" `dbt compile --select int_ga4__sessionized int_ga4__funnel_steps` (ephemeral models don't materialize standalone, so compile is how you confirm refs + macros resolve). Then `dbt build --select +fct_funnel --full-refresh` and confirm `assert_funnel_monotonicity` returns 0 rows. A BigQuery dry_run on the compiled SQL catches any hallucinated column not in `stg_ga4__events`.
- **R4 (canonical names)** â€" `grep -n "channel_group" models/intermediate/*.sql` resolves to the macro call; no inline CASE. `grep "did_"` clean (modulo `did_session_start`). The `accepted_values` test on `channel_group` lists exactly the 10 groups.

Then **`/clear` before M4** â€" the intermediates are done; carrying their context into M4 only adds drift and burns budget.

M4 â€" Marts [Wk1-MVP]

M4 assembles the **consumption layer** â€" the only models the semantic layer (and therefore the agents) may read. The keystones did the hard logic; marts *assemble, they do not re-derive*. The job is wide, conformed, grain-explicit facts: `fct_sessions` and `fct_funnel` (session grain, incremental), `fct_daily_funnel` (additive day Ã— dims, incremental), `fct_orders` / `fct_order_items` (finance, table), `dim_channels` / `dim_users` / `dim_items` / `dim_date` (table), and `fct_cohorts` / `fct_funnel_by_dim` (growth, table). The two machine-checks that define "done" are **`revenue_reconciles` to the cent** and **`dim_channels` `accepted_values` = exactly 10**; everything else is uniqueness, referential integrity, and a clean dry_run.

Context to load

- **Root `CLAUDE.md`** (auto-loads) â€" the WIDE-marts rule, the five base grains (`fct_funnel`, `fct_sessions`, `fct_orders`, `fct_order_items`, `fct_cohorts`), revenue = `*_in_usd`, rates computed `SUM(num)/SUM(den)` *in the semantic layer* (never stored in a mart).
- **`@docs/architecture/DBT_GUIDE.md` Â§5** â€" the full mart catalog (Â§5.1), why WIDE (Â§5.2), the incremental config (Â§5.3), per-mart grain/columns (Â§5.4â€"5.6), and the copy-usable SQL: Â§5.8 `fct_funnel`, Â§5.9 `fct_daily_funnel`, Â§5.10 `fct_orders`, Â§5.11 `dim_channels`, Â§5.12 `fct_cohorts`, plus the schema.yml tests (Â§5.13).
- **`@docs/architecture/DBT_GUIDE.md` Â§6.4** â€" the custom generic test `test_revenue_reconciles` (mart total == raw `purchase_revenue_in_usd`, tolerance 0).
- **`@docs/architecture/DATA_MODEL.md` Â§3â€"8** â€" the grains as spec: Â§5.7 `fct_sessions`, Â§5.8 `fct_funnel` (1:1 with `fct_sessions`), Â§5.9 `fct_daily_funnel` (additive counts, rates NOT stored), the order/item grains, and the cohort grain.
- The **upstream models** for each mart: `@models/intermediate/int_ga4__sessionized.sql` + `@models/intermediate/int_ga4__funnel_steps.sql` (feed `fct_funnel` / `fct_sessions`), `@models/marts/core/fct_funnel.sql` (feeds `fct_daily_funnel` â€" NOT the intermediate), `@models/staging/stg_ga4__events.sql` (feeds `fct_orders`), `@macros/channel_group.sql` (the 10-group source of truth `dim_channels` enumerates).

Run M4 in a **fresh session**. Plan mode first: confirm the build order `fct_sessions â†' fct_funnel â†' fct_daily_funnel`, `dim_*` independently, `fct_orders/items` after `fct_sessions`, `fct_cohorts` after `fct_sessions`.

Files to provide

Attach	Why
`@docs/architecture/DBT_GUIDE.md` Â§5.8â€"5.13	the copy-usable mart SQL + schema.yml tests (D2)
`@docs/architecture/DBT_GUIDE.md` Â§6.4	the `test_revenue_reconciles` generic test
`@docs/architecture/DATA_MODEL.md` Â§5.7â€"5.9	grain/PK/FK/column spec for each fact

Attach	Why
`@models/intermediate/int_ga4__sessionized.sql`, `@models/intermediate/int_ga4__funnel_steps.sql`	the real upstream columns `fct_funnel` / `fct_sessions` ref (D3)
`@models/marts/core/fct_funnel.sql`	the upstream of `fct_daily_funnel` (it aggregates this, NOT the intermediate)
`@macros/channel_group.sql`	the 10 canonical groups `dim_channels` must enumerate exactly

Prompts to use

Prompt 1 â€” the conformed dim + the session facts, with their tests:

```
[ROLE] You are implementing Helios milestone M4, files models/marts/core/dim_channels.sql,
  fct_sessions.sql, fct_funnel.sql and models/marts/core/core__schema.yml.
[SOURCE OF TRUTH] Copy/adapt @docs/architecture/DBT_GUIDE.md Â§5.11 (dim_channels), Â§5.8 (fct_funnel),
  and Â§5.4 (fct_sessions columns); tests from Â§5.13 (core__schema.yml). Cross-check grain/PK/FK
  against @docs/architecture/DATA_MODEL.md Â§5.7â€“5.8. Follow CLAUDE.md (auto-loaded).
[UPSTREAM] Reference only @models/intermediate/int_ga4__sessionized.sql and
  @models/intermediate/int_ga4__funnel_steps.sql; the 10 groups in dim_channels are the exact
  strings in @macros/channel_group.sql.
[TASK] dim_channels = exactly 10 rows (Direct, Organic Search, Paid Search, Display, Paid Social,
  Organic Social, Email, Affiliates, Referral, Other) with is_paid/is_organic/channel_group_order
  and channel_key = to_hex(md5(channel_group)). fct_sessions + fct_funnel: session grain (PK
  session_key), WIDE denormalized dims, fct_funnel joins the two intermediates 1:1 USING(session_key)
  and carries the reached_* flags + session_revenue. Use the incremental insert_overwrite config
  from Â§5.3 (partition_by event_date, cluster_by [device_category, channel_group],
  require_partition_filter=true).
[CONSTRAINTS] WIDE marts â€” no rates stored (session_conversion_rate etc. live ONLY in the semantic
  layer). channel_group strings must match channel_group_case() exactly â€” exactly 10, no 11th group,
  no "Paid Other". Use ONLY columns present in the intermediates. If one is missing, STOP and ASK.
[TEST-FIRST] Add the schema.yml tests (unique/not_null PK, channel_group accepted_values = the 10,
  relationships fct_funnel.channel_group → dim_channels), then
  `dbt build --select dim_channels fct_sessions fct_funnel --full-refresh` and show the test output.
[OUTPUT] Cite Â§5.8/Â§5.11/Â§5.13 per block.
```

Prompt 2 â€” the additive daily rollup + the finance fact with the reconcile test:

```
[ROLE] You are implementing Helios milestone M4, files models/marts/core/fct_daily_funnel.sql,
  models/marts/finance/fct_orders.sql, finance__schema.yml, and the generic test
  tests/generic/test_revenue_reconciles.sql.
[SOURCE OF TRUTH] Copy/adapt @docs/architecture/DBT_GUIDE.md Â§5.9 (fct_daily_funnel), Â§5.10 (fct_orders),
  Â§6.4 (test_revenue_reconciles), tests from Â§5.13. Cross-check @docs/architecture/DATA_MODEL.md Â§5.9.
  Follow CLAUDE.md (auto-loaded).
[UPSTREAM] fct_daily_funnel aggregates @models/marts/core/fct_funnel.sql (which carries
  session_revenue) â€” NOT int_ga4__funnel_steps. fct_orders reads @models/staging/stg_ga4__events.sql
  (purchase events) + the wide dims from fct_sessions.
[TASK] fct_daily_funnel: grain event_date Ã— [channel_group, device_category, country, is_new_user],
  daily_funnel_key = md5 of grain, additive counts only (sessions, view_item_sessions ...
  purchasing_sessions, transactions, revenue=SUM(session_revenue)) â€” NO rates. fct_orders: one
  deduped row per transaction_id (PK order_key), any_value() per txn, net_revenue = gross − refund,
  wide channel/device/date dims. Materialize fct_orders as table; fct_daily_funnel incremental.
[CONSTRAINTS] fct_daily_funnel MUST ref fct_funnel (revenue must survive). Aggregate only *_in_usd
  revenue. Do NOT store any rate. Use ONLY canonical column names; if absent, STOP and ASK.
[TEST-FIRST] Wire test_revenue_reconciles on fct_orders.gross_revenue (tolerance 0) and on
  fct_funnel.session_revenue, then `dbt build --select fct_funnel fct_daily_funnel fct_orders
  --full-refresh` and show that revenue_reconciles passes and purchasing_sessions ≤ sessions holds.
[OUTPUT] Cite Â§5.9/Â§5.10/Â§6.4 per block.
```

Preventing hallucinations

- **`fct_daily_funnel` MUST aggregate `fct_funnel`, not `int_ga4__funnel_steps` (D2/D3).** This is the single most common M4 drift. `fct_funnel` carries `session_revenue`; the intermediate funnel-steps model does too, but the spec (Â§5.9, CLAUDE.md keystone #5) pins `fct_funnel` as the source so the dollar-at-risk labels the eval needs are preserved. Verify the `ref()`: `grep -n "ref("` `models/marts/core/fct_daily_funnel.sql` must show `ref('fct_funnel')`.
- **`dim_channels` is exactly 10 rows, enumerated to match the macro (D7).** Reject any 11th group, "Paid Other", or a string that disagrees with `channel_group_case()` in `@macros/channel_group.sql`. The `accepted_values` test in Â§5.13 lists the 10 â€” they must match the macro's emitted strings character-for-character.
- **WIDE, no stored rates (D5).** If Claude adds `session_conversion_rate`, `aov`, or any ratio column to a mart, reject it â€” rates live only in the semantic layer (M5), computed `SUM(num)/SUM(den)` after grouping. Marts store additive primitives.

- **Let the reconcile + dry_run catch invented columns (D6).** `test_revenue_reconciles` (Â§6.4) sums the mart's revenue against raw `purchase_revenue_in_usd` and fails on >tolerance drift â€" a hallucinated/double-counted revenue column surfaces here. A BigQuery dry_run on each compiled mart fails on any column not present upstream and prints the byte estimate (keep it inside the 5 GiB run budget).
- **Grain is explicit, one entity per mart.** `fct_funnel` = session, `fct_orders` = transaction, `fct_order_items` = order line. If a generated mart mixes grains (e.g. order columns on the session fact), reject it.

Reviewing generated code

- **R3/R6 â€" reconcile to the cent, then dry_run.** The acceptance gate is `dbt build --select fct_orders fct_funnel --full-refresh` showing `revenue_reconciles` green (mart `gross_revenue` total == raw to the cent, tolerance 0 on `fct_orders`; 0.5% on `session_revenue`). Then dry_run each mart for hallucinated columns + cost.
- **R4 â€" the 10-channel closed set.** `dbt test --select dim_channels` must pass `accepted_values` = exactly the 10 groups, and `unique` + `not_null` on `channel_key` / `channel_group` (10 rows, no dupes, no 11th). Confirm the `relationships` test from `fct_funnel.channel_group` / `fct_orders.channel_group` â†' `dim_channels` passes (referential integrity of the conformed dim).
- **R5 â€" read the diff by eye for two things.** (1) `fct_daily_funnel` refs `fct_funnel` (not the intermediate) and selects `SUM(session_revenue) as revenue` â€" revenue must survive the rollup. (2) The session facts are WIDE (dims denormalized onto the row) and store no rate column. Confirm the incremental config block (`insert_overwrite`, `partition_by` `event_date`, `cluster_by` `[device_category, channel_group]`, `require_partition_filter=true`) matches Â§5.3.
- **R2 â€" monotonicity at the rollup grain.** `fct_daily_funnel` carries the `dbt_utils.expression_is_true` chain (`purchasing_sessions ≤ … ≤ view_item_sessions ≤ sessions`); confirm it is present and green â€" it is the additive-grain twin of the keystone monotonicity test.

Then `/clear` **before M5** (semantic layer) â€" the marts are now the contract; M5 reads `semantic_layer.yaml` + the marts, and a clean context keeps the registry validation focused.

M5 - Semantic Layer live [Wk1-MVP]

The registry `models/semantic/semantic_layer.yaml` (registry v2.0.0, 47 metrics / 19 dimensions, descriptive field names `metric_name` / `sql_definition` / `aggregation_method` / `dimensions_supported`) **already exists and is RI-clean**. Claude Code's job is NOT to author metrics. It is to (1) wrap the registry in a thin dbt exposure (`metrics__schema.yml`) so it joins the dbt DAG, and (2) write `scripts/validate_semantic.py`, the standalone compile gate that proves every `numerator` / `denominator`, every derived `expr {token}`, every `dimensions_supported` entry, every `agents[]` entry, and every `grain` resolves against the live registry. The validator is the M5 exit gate (METRIC_GOVERNANCE_GUIDE.md Â§6.1; invariant I1).

Filename-drift to fix in this milestone. Two files sit on disk: `semantic_layer.yaml` (v2.0.0, 47 metrics â€" **canonical**) and the retired `semantic_models.yml` (v1, ~28 metrics). MCP_ARCHITECTURE.md Â§3 and CLAUDE.md Â§5 still cite `semantic_models.yml`. Point the validator (and later `mcp_servers.yaml`) at `semantic_layer.yaml`. Have Claude confirm which file before writing a line â€" if it validates the v1 file, the gate is green but governs the wrong layer.

Context to load

- CLAUDE.md (auto-loads at repo root â€" pins canonical names, `reached_*`, `SUM(num)/SUM(den)`, the 10 channel groups; D1).
- `@docs/architecture/METRIC_GOVERNANCE_GUIDE.md` â€" sections Â§2 (per-metric field contract), Â§2.2 (typeâ†'population matrix), Â§6.1 (referential-integrity compile = exactly what the validator must enforce), Â§9.1 (canonical dimension list, 6 valid agents).
- `@models/semantic/semantic_layer.yaml` â€" the live registry. Open it so Claude reads the **actual field names** (`metric_name`, `sql_definition`, `dimensions_supported`, `agents`) and the real metric set, not what it remembers (D2/D3).
- The milestone tracker / `/resume` output for the next-step pin (D8: fresh session, M5 only).

Files to provide

File	Why
`@models/semantic/semantic_layer.yaml`	Source of truth â€" Claude reads its keys to write field-accurate validation; never regenerated.
`@docs/architecture/METRIC_GOVERNANCE_GUIDE.md` (Â§2, Â§2.2, Â§6.1, Â§9.1)	The spec the validator encodes; cite per check.
`@models/semantic/metrics__schema.yml`	Once it exists, attach for the exposure edit.
`@scripts/validate_semantic.py`	Once it exists, attach for iteration.

Prompts to use

Plan first (D9):

```
You are implementing Helios milestone M5. Plan mode only â€” do NOT write code yet.
The registry @models/semantic/semantic_layer.yaml ALREADY EXISTS (registry v2.0.0, 47 metrics).
Do NOT regenerate or edit it. First: open it and tell me the EXACT top-level keys and the EXACT
per-metric field names it uses (metric_name? name? sql_definition? sql? dimensions_supported? agents?),
and confirm it is the v2.0.0 file (not the retired semantic_models.yml). Then propose:
 (1) models/semantic/metrics__schema.yml â€” a dbt exposure depending on the five mart grains, and
 (2) scripts/validate_semantic.py â€” a standalone, no-network, no-warehouse validator enforcing the
     METRIC_GOVERNANCE_GUIDE Â§6.1 compile (ratio num/den resolve, derived expr {token}s resolve,
     dimensions_supported all defined, agents[] in {Monitor,Decompose,Diagnose,Prescribe,Narrator,Critic},
     grain in grains:, and the Â§2.2 typeâ†'population matrix).
Cite the Â§6.1 bullet each check implements. Show me the plan before any edit.
```

Then implement against the confirmed field names:

```
[ROLE] Implement Helios M5, file scripts/validate_semantic.py.
[SOURCE OF TRUTH] Encode the METRIC_GOVERNANCE_GUIDE Â§6.1 referential-integrity compile and the Â§2.2
typeâ†'population matrix. Follow CLAUDE.md conventions (auto-loaded).
[UPSTREAM] Read field/metric names ONLY from @models/semantic/semantic_layer.yaml â€" use the EXACT field
names you reported in the plan (the v2 file uses descriptive names: metric_name, sql_definition,
aggregation_method, dimensions_supported, agents, grain).
[TASK] Pure-Python validator: load the YAML, collect metric_names/dimension_names/grain keys, then assert
 (1) every ratio numerator+denominator is a defined metric_name; (2) every derived expr {token}
 (regex \{([a-zA-Z0-9_]+)\}) is a defined metric_name; (3) every dimensions_supported entry is a defined
 dimension; (4) every agents[] entry â^^ {Monitor,Decompose,Diagnose,Prescribe,Narrator,Critic};
 (5) every grain â^^ grains:; (6) Â§2.2 matrix (count/sumâ†'sql_definition set, num/den/expr null;
 ratioâ†'num+den set, expr null; derivedâ†'expr set, num/den null). Accumulate ALL errors; on any error
 print "FAIL: <n> dangling/invalid reference(s)" naming each offender and exit 1; else print
 "PASS: <m> metrics, <d> dimensions — 0 dangling references" and exit 0. No network, no warehouse, no AI.
[TEST-FIRST] Before the validator, write tests/test_validate_semantic.py with: a PASS case on the real
 registry, and a NEGATIVE case that copies the registry to a temp file, corrupts one ratio denominator
 to 'sessionz', and asserts exit 1 + the offending name in output.
[CONSTRAINTS] Point REGISTRY at models/semantic/semantic_layer.yaml (NOT semantic_models.yml). Do NOT
 invent metric/dimension names; read them from the file. If a field name is not what you expected, STOP and ASK.
[OUTPUT] Run `python scripts/validate_semantic.py` and `pytest tests/test_validate_semantic.py -v`; paste
 both outputs. Cite the Â§6.1 bullet each check implements.
```

Exposure wrapper (small, can share the same session):

```
[ROLE] Implement Helios M5, file models/semantic/metrics__schema.yml.
[SOURCE OF TRUTH] METRIC_GOVERNANCE_GUIDE Â§1 (single-source-of-truth) + CLAUDE.md Â§5 (semantic layer is
a view; the YAML is the governance object).
[TASK] A dbt v2 exposure named semantic_layer (type: application, maturity: high, owner analytics-eng)
that depends_on ref('fct_funnel'), ref('fct_sessions'), ref('fct_orders'), ref('fct_order_items'),
ref('fct_cohorts'). Description: it is the single source of truth for all 47 metrics / 19 dimensions
that semantic-mcp.build_query reads, RI-enforced by scripts/validate_semantic.py.
[TEST] Run `dbt parse` and paste output â€" the exposure must register with no ref errors.
[OUTPUT] Cite the doc section.
```

Preventing hallucinations

- D2/D3 specialized: the registry is the upstream. Make Claude **read the real field names from** `semantic_layer.yaml` before coding â€" the v2 file uses `metric_name` / `sql_definition` / `dimensions_supported`, not `name` / `sql` / `dimensions`. If Claude codes against remembered keys it will silently match nothing and print a false PASS.
- D5: forbid inventing metric names in `REQUIRED_METRICS`. If you include a floor of must-exist names, derive them from the open registry, not from memory â€" a non-existent "required" name turns the gate into a false FAIL.
- D6 specialized: the validator IS the machine that catches hallucinated refs. Treat any dangling `numerator` / `denominator` / `{token}` /dimension/grain as a **hard fail** â€" never "fix" it by editing `semantic_layer.yaml`; fix the typo or the validator. Editing a governed definition to satisfy a tool is backwards (the registry is RI-clean by construction).
- Extract `expr` tokens with the `\{ … \}` regex, never a comma split: `SAFE_DIVIDE({revenue},{users})` has two tokens, both must resolve.
- Reject `Orchestrator` in any `agents[]` (it plans, consumes no metrics) â€" only the 6 in the valid set are legal (Â§9.1).

Reviewing generated code

- R1: each check cites the Â§6.1 bullet it implements.
- R3 (the load-bearing check): Claude RAN the validator AND the negative test. `python scripts/validate_semantic.py; echo exit=$?` â†' `PASS: 47 metrics, 19 dimensions — 0 dangling references`, `exit=0`. The negative test must print `FAIL` naming `denominator 'sessionz' is not a defined metric_name` and exit 1. A gate that cannot fail is not a gate â€" reject any PR without the proven-FAIL test.
- R2: `dbt parse` succeeds; the `semantic_layer` exposure depends on the five grains with no ref error.

- R4: confirm REGISTRY path = `semantic_layer.yaml` (grep the script). If it loaded `semantic_models.yml`, the metric count prints ~28/44 and the gate governs the retired layer â€" reject.
- R5: typeâ†'population matrix actually enforced (a ratio with a stray `expr`, or a derived with `numerator` set, is flagged even when names resolve).

M6 - Grounding MCP pair [Wk2-v1: build ONE server]

Full design = `semantic-mcp` (only path to SQL) + `warehouse-mcp` (sole BigQuery client, dry-run + byte-budget gate). **LEAN (Wk2-v1): build ONE server.** Pick `semantic-mcp` (`build_query` from the registry) â€" it carries the grounding story ("the model composes governed metrics, never writes raw SQL â†' 0 hallucinated columns") and is data-independent enough to demo without a live warehouse round-trip. (M6b covers the stats alternative.) The critical anti-hallucination property is **structural**: `build_query` interpolates only registry templates, so a column not in the registry has no surface to enter through (invariant I3). Connect the server to Claude Code so Claude can ONLY get data through the governed tool.

Context to load

- CLAUDE.md (auto: MCP server/tool table, grounding rules G1/G5).
- `@docs/architecture/MCP_ARCHITECTURE.md` â€" Â§6.2 (semantic-mcp tool I/O), Â§7 (registry binding = the anti-hallucination core), Â§8 (compile-time integrity, fail-loud), Â§9 (the `build_query` resolver skeleton â€" copy/adapt), Â§5 (error taxonomy), Â§11 (semantic-mcp tests).
- `@docs/architecture/METRIC_GOVERNANCE_GUIDE.md` Â§3 (the authoringâ†"resolver field-name mapping â€" `metric_nameâ†'name`, `sql_definitionâ†'sql`, `aggregation_methodâ†'agg`, `dimensions_supportedâ†'dimensions`) â€" load-bearing: the v2 YAML is authored in descriptive names, the resolver reads short keys.
- `@models/semantic/semantic_layer.yaml` (the registry the server loads + RI-compiles at startup).
- The expected golden SQL shape from MCP_ARCHITECTURE Â§6.2 (the `session_conversion_rate Ã— device_category` snapshot).

Files to provide

File	Why
`@docs/architecture/MCP_ARCHITECTURE.md` (Â§5, Â§6.2, Â§7, Â§8, Â§9, Â§11)	Skeletons + contract + tests; Claude copies/adapts, not invents.
`@models/semantic/semantic_layer.yaml`	The registry the resolver reads; real metric/dim/grain names.
`@docs/architecture/METRIC_GOVERNANCE_GUIDE.md` Â§3	The authoringâ†'resolver field mapping the resolver must apply at load.
`@helios/mcp/base.py`, `@helios/mcp/schemas.py`	Once they exist, attach for the semantic.py session.
`@mcp_servers.yaml`	The registration/config â€" set `registry: ./models/semantic/semantic_layer.yaml`.

Prompts to use

Plan + the field-mapping trap first (D9):

```
You are implementing Helios M6 (LEAN: semantic-mcp ONLY). Plan mode â€" no code yet.
Source of truth: MCP_ARCHITECTURE Â§6.2/Â§7/Â§8/Â§9. The registry is @models/semantic/semantic_layer.yaml
(v2.0.0). CRITICAL: that file is authored in DESCRIPTIVE field names (metric_name, sql_definition,
aggregation_method, dimensions_supported) but the Â§9 resolver reads SHORT keys (name, sql, agg, dimensions)
â€" METRIC_GOVERNANCE_GUIDE Â§3 defines the mapping. Confirm by opening the file, then propose:
 - base.py: typed errors (UnknownMetric -32001, UnknownDimension -32002, DimensionNotPermitted -32003,
   InvalidFilter -32004) + normalize_hash(sql);
 - semantic.py: load + RI-compile the registry at startup (fail loud per Â§8 if any dangling ref),
   APPLY the Â§3 field mapping on load, serve get_metric/list_dimensions/build_query per the Â§9 resolver.
Tell me where you will apply the metric_nameâ†'name mapping and how a ratio composes
SAFE_DIVIDE(SUM(num),SUM(den)) (not AVG of ratios). Show the plan; cite Â§6.2/Â§7/Â§8/Â§9 per piece.
```

Implement the resolver (copy/adapt the Â§9 skeleton):

```
[ROLE] Implement Helios M6, file helios/mcp/semantic.py (LEAN: semantic-mcp only).
[SOURCE OF TRUTH] Copy/adapt the build_query resolver in MCP_ARCHITECTURE Â§9; obey the Â§7 binding rules
and Â§8 fail-loud compile. Apply the METRIC_GOVERNANCE_GUIDE Â§3 field mapping at load
(metric_nameâ†'name, sql_definitionâ†'sql, aggregation_methodâ†'agg, dimensions_supportedâ†'dimensions).
[UPSTREAM] Read metric/dimension/grain names ONLY from @models/semantic/semantic_layer.yaml. Errors come
from @helios/mcp/base.py.
[TASK] On startup: parse the registry, build metrics/dimensions/grains dicts, run the Â§8 RI compile and
 refuse to start on ANY dangling ref. Serve: get_metric(name); list_dimensions(); build_query(metric|[metric],
```

```
 dims, filters, window) â†’ governed SQL. _measure(): count/sum â†’ AGG(sql) AS name; ratio â†’
 SAFE_DIVIDE(SUM(num_expr),SUM(den_expr)) AS name (SUM-of-num/SUM-of-den AFTER grouping, NEVER AVG of
 per-row ratios); derived â†’ expand_expr(expr) AS name. For countif(reached_*) measures emit the
 session-keyed subquery form shown in Â§6.2. Validate each dim against the metric's dimensions list
 (DimensionNotPermitted) and each name against the registry (UnknownMetric/UnknownDimension). This server
 has NO BigQuery client and emits SQL only â€” it cannot execute.
[TEST-FIRST] Write helios/mcp/tests/test_semantic.py first: (a) UnknownMetric/UnknownDimension â†’ raised,
 NO SQL string returned; (b) build_query('session_conversion_rate',['device_category'],window='last_28d')
 snapshot-matches the Â§6.2 governed SQL (session-keyed subquery, SAFE_DIVIDE(COUNTIF( ... ),COUNT(DISTINCT
 session_key))); (c) a copy of the registry with one dangling numerator â†’ server refuses to start;
 (d) a non-whitelisted dim â†’ DimensionNotPermitted.
[CONSTRAINTS] The model passes ONLY string names; physical columns live exclusively in registry sql fields
 (I3). Use ONLY names in the attached registry. If a name is missing, STOP and ASK â€” never fall back to free SQL.
[OUTPUT] Run `pytest helios/mcp/tests/test_semantic.py -v` and paste output. Cite Â§6.2/Â§7/Â§9 per block.
```

Register + smoke-test the server with Claude Code (the grounding demo):

```
[ROLE] Wire semantic-mcp into this Claude Code session and prove I can only query through it.
[TASK] 1) Set mcp_servers.yaml: semantic-mcp transport stdio, command ["python","-m","helios.mcp.semantic"],
 config.registry ./models/semantic/semantic_layer.yaml (NOT semantic_models.yml). 2) Register it for THIS
 session (show me the exact `claude mcp add` command and the .mcp.json entry):
     claude mcp add semantic-mcp -- python -m helios.mcp.semantic
 3) After /mcp shows it connected, smoke-test by calling its tools (not by writing SQL yourself):
   - list_dimensions() â†’ confirm the 19 canonical dims;
   - get_metric("session_conversion_rate") â†’ confirm numerator=purchasing_sessions, denominator=sessions;
   - build_query("session_conversion_rate", ["device_category"], window="last_28d") â†’ paste the governed SQL;
   - build_query("session_conversion_rate", ["weather"]) â†’ confirm UnknownDimension (hard fail, no SQL).
[CONSTRAINTS] Do NOT hand-write any SQL. The ONLY way you may obtain a query string is the build_query tool.
 If a name is rejected, re-plan against list_dimensions() â€” do not invent a synonym (G5).
[OUTPUT] Paste the /mcp connection status and each tool call's result.
```

Preventing hallucinations

- The structural guard (I3): `build_query` emits ONLY registry templates, so a hallucinated column (`event_params.foo`) literally cannot appear. Verify Claude did not add a free-SQL escape hatch â€” there is no path from a string name to a raw column except via the registry `sql` field.
- The Â§3 field-mapping trap (D2/D3): if the resolver reads `m["sql"]` against the v2 file (which stores `sql_definition`), every measure is empty/KeyError. Make Claude apply the mapping at load and prove it with the Â§6.2 snapshot test.
- D5/G5: an unknown metric/dimension is a HARD error (`UnknownMetric` / `UnknownDimension`), never a fallback to free SQL. Test the `['weather']` rejection.
- Â§8 fail-loud (D6): a registry with a dangling ref â†’ the server **refuses to start**. Never serve a half-valid registry. (M5's `validate_semantic.py` is the same compile run standalone â€” keep both green.)
- Registry filename in `mcp_servers.yaml` MUST be `semantic_layer.yaml` ; pointing at `semantic_models.yml` silently loads the retired 28-metric layer and "works," wrongly.
- Simpson's defense lives in the resolver: ratios are `SAFE_DIVIDE(SUM(num),SUM(den))` after grouping, never `AVG(per_row_ratio)` .

Reviewing generated code

- R1: resolver blocks cite Â§9; field mapping cites Â§3; fail-loud cites Â§8.
- R3: the Â§6.2 snapshot test ran and matches â€” the emitted SQL for `session_conversion_rate Ã— device_category` is the session-keyed subquery form with `SAFE_DIVIDE(COUNTIF(purchased),COUNT(DISTINCT session_key))` . This is the contract test that proves grounding.
- R4 (AST/grep check, shared with the eval scorer per Â§11): parse the emitted SQL; every column/table must be in the registry or GA4 schema â€” any other name is a hard-zero fail.
- R5: ratios emit `SUM(num)/SUM(den)` after grouping (grep for `AVG(` â†’ must be absent in rate composition); `reached_*` flags compose via the session-keyed subquery so step rates stay â‰¤ 1.
- Grounding wiring proof: `/mcp` shows `semantic-mcp` connected, and Claude obtained every query via `build_query` â€” confirm no hand-written SQL appears in the transcript, and the `['weather']` call returned `UnknownDimension` with NO SQL.
- If you instead built `warehouse-mcp` (the other half), additionally verify R6: `run_query` without a prior `dry_run` → `NotDryRunFirst` ; `max_bytes_billed` capped at 5 GiB → `ByteBudgetExceeded` on over-scan; `reconcile` within 0.5% of a hand control query; a write SQL → error (read-only SA). The hash must normalize identically (whitespace-collapsed, lowercased) in `dry_run` and `run_query` or the gate spuriously fails.

M6b - stats / experiment / report MCP [Wk2-v1: the chosen server only; rest v2]

Full design = three data-independent servers: `stats-mcp` (only path to math, seeded), `experiment-mcp` (powered test cards), `report-mcp` core (`render_brief` / `export`). **LEAN: build the ONE server you chose.** If you built `semantic-mcp` in M6, the natural M6b pick is **`stats-mcp`** for `decompose_change` â€" the thesis centerpiece that dissolves Simpson's paradox by splitting `Î”R` into mix/rate/interaction. It **fails silently** (wrong numbers, not an error), so it gets a GOLDEN test FIRST. `experiment-mcp` and `report-mcp` memory are **[v2-later]**; a one-line "recommended next test" in the brief replaces `design_experiment` for the lean cut (LEAN_SCOPE).

Context to load

- CLAUDE.md (auto: the decomposition identity verbatim, `rng_seed: 1729` , G2 "never compute a statistic in prose").
- `@docs/architecture/MCP_ARCHITECTURE.md` â€" Â§6.3 (stats tool I/O), Â§9 (the `decompose_change` skeleton â€" copy verbatim), Â§11 (the GOLDEN test: `mix=-0.0018, rate=0, interaction=0, delta_R=-0.0018 Â±1e-9`), Â§5 (errors: `SegmentMismatch` , `InsufficientData`).
- `@models/semantic/semantic_layer.yaml` â€" only if you also wire `experiment-mcp.design_experiment` (it validates target/guardrail metrics against the registry); otherwise not needed (stats has no data access).

Files to provide

File	Why
`@docs/architecture/MCP_ARCHITECTURE.md` (Â§6.3, Â§9, Â§11, Â§5)	`decompose_change` skeleton + the golden values; copy verbatim.
`@helios/mcp/base.py`	Typed errors (`SegmentMismatch` -32031, `InsufficientData` -32030, `ZeroSample` -32032).
`@helios/mcp/stats.py`	Once it exists, for iteration.
`@models/semantic/semantic_layer.yaml`	Only if building `experiment-mcp` (registry validation of test-card metrics).

Prompts to use

Golden test FIRST, then the implementation (TDD; the silent-failure guard):

```
[ROLE] Implement Helios M6b, file helios/mcp/stats.py (LEAN: stats-mcp; decompose_change is the keystone).
[SOURCE OF TRUTH] Copy the decompose_change skeleton from MCP_ARCHITECTURE Â§9 VERBATIM â€" it implements the
FOUNDATION identity exactly: mix = Î£ Î”wÂ·r(t0), rate = Î£ w(t0)Â·Î”r, interaction = Î£ Î”wÂ·Î”r,
Î”R = mix + rate + interaction. Errors from @helios/mcp/base.py. Seed rng_seed=1729 at import.
[TEST-FIRST] BEFORE the code, write helios/mcp/tests/test_decompose_golden.py with the Â§11 golden case:
   t0 = [{"seg":"desktop","w":0.40,"r":0.030},{"seg":"mobile","w":0.60,"r":0.012}]
   t1 = [{"seg":"desktop","w":0.30,"r":0.030},{"seg":"mobile","w":0.70,"r":0.012}]   # weights shift, rates flat
   assert mix_effect == -0.0018, rate_effect == 0, interaction == 0, delta_R == -0.0018  (all Â±1e-9)
 Also: identity mix+rate+interaction == delta_R on 100 random seeded inputs; SegmentMismatch when t0/t1
 segment sets differ; determinism (same seed â†' byte-identical output).
[TASK] Implement decompose_change(metric, dim, t0, t1) per Â§9 (raise SegmentMismatch on differing segment
 sets). Add significance_test(a,b,kind="proportion"|"mean") matching a scipy reference. (detect_anomaly/
 forecast/cohort_retention/rfm_segment are v2 â€" stub or omit per LEAN.) NO data access; NO warehouse client.
[CONSTRAINTS] mix uses r(t0); rate uses w(t0). Do NOT swap the t0/t1 baselines â€" it flips attribution and the
 golden test is the only thing that catches it. Numbers are tool outputs; the LLM never does this arithmetic (G2).
[OUTPUT] Run `pytest helios/mcp/tests/ -v` and paste output (golden + identity + determinism must pass).
 Cite Â§9/Â§11.
```

Register + smoke-test stats-mcp with Claude Code:

```
[ROLE] Wire stats-mcp into Claude Code and prove the decomposition is a tool, not prose.
[TASK] 1) mcp_servers.yaml: stats-mcp transport stdio, command ["python","-m","helios.mcp.stats"],
 config.rng_seed 1729. 2) Register for this session:
     claude mcp add stats-mcp -- python -m helios.mcp.stats
 3) After /mcp shows connected, call decompose_change with the Â§11 golden inputs (desktop/mobile,
 weights 0.40/0.60 â†' 0.30/0.70, rates 0.030/0.012 unchanged). Paste the result and confirm
 mix_effect=-0.0018, rate=0, interaction=0. 4) Re-run identical input twice â†' identical output (seed 1729).
[CONSTRAINTS] Do NOT compute mix/rate/interaction yourself in text (G2). The ONLY source of the numbers is
 the tool call. State in one line what this result MEANS: the blended rate fell purely because traffic
 shifted to lower-converting mobile â€" mix-shift, not behavior change.
[OUTPUT] Paste /mcp status, the tool result, and the two-run determinism check.
```

(If `experiment-mcp` is your chosen server instead, add: `design_experiment(hypothesis, metric)` must validate `metric` and every guardrail metric against `@models/semantic/semantic_layer.yaml` â€" a card can only target a governed metric like `net_revenue` , never `refund_rate` ; test that an ungoverned name â†' `UnknownMetric` .)

Preventing hallucinations

- G2 specialized: every statistic is a `stats-mcp` output, never prose. In the smoke test, forbid Claude from hand-computing mix/rate in text â€" the numbers must come from the tool call.
- The silent-failure guard (D6): `decompose_change` returns wrong numbers, not an error, when the formula is off. The Â§11 golden test (`mix=-0.0018`) is the ONLY thing that catches a swapped `t0` / `t1` baseline â€" write it FIRST and make it pass before trusting any output.
- Copy the Â§9 skeleton verbatim: `mix` uses `r(t0)` , `rate` uses `w(t0)` . Adapting the algebra from memory is the classic drift here.
- Determinism (seed 1729): seed at import so the byte-identical test passes and eval grading is reproducible. Non-determinism breaks both.
- `SegmentMismatch` is not ignorable: if `t0` / `t1` segment sets differ the decomposition is undefined (mix double-counts appearing/disappearing segments) â€" raise, don't paper over.
- `report-mcp` (if built) gets NO BigQuery client â€" rendering is pure string work; `render_brief([])` â†' `EmptyFindings` (emit a "no material change" brief, never fabricate a finding). Memory tools (`save_diagnosis` / `recall_prior`) are [v2-later], not M6b.

Reviewing generated code

- R3 (the keystone check): the GOLDEN test ran and passed exactly â€" `mix_effect=-0.0018, rate_effect=0, interaction=0, delta_R=-0.0018` (Â±1e-9) on the Â§11 example. No PR without this passing.
- R1: `decompose_change` cites Â§9; the golden values cite Â§11.
- R5: verify `mix` uses `r(t0)` and `rate` uses `w(t0)` by READING the diff against the Â§9 formulas â€" a swapped baseline still satisfies `mix+rate+interaction == delta_R` (the identity check passes!) yet attributes wrongly; only the golden test distinguishes them.
- R2: determinism â€" re-run any stats tool on the same input twice; outputs byte-identical (seed 1729). `significance_test` matches a scipy reference within tolerance.
- Wiring proof: `/mcp` shows `stats-mcp` connected; the golden result came from the tool call (in the transcript), not Claude's arithmetic.
- For `experiment-mcp` (if chosen): R4 â€" `design_experiment` rejects an ungoverned metric (`UnknownMetric`); `power_analysis` two-proportion sample size matches the closed-form reference. For `report-mcp` : R6-style â€" `render_brief([])` â†' `EmptyFindings` , and every number in a brief traces to a backing tool-output hash (faithfulness rule, Â§11).

M7 - Minimal Loop: ONE Grounded LLM Brief [Wk2-v1]

This is the v1 centerpiece and the **only** place you deliberately spend the LLM budget. Per LEAN_SCOPE, M7 is NOT the 7-agent FSM (Monitor->Decompose->Diagnose->Critic->Prescribe->Narrator). The lean M7 is a **single grounded LLM call**: a deterministic Python step computes the decomposition + significance + dollar revenue-at-risk (the M5/M6b code you already built and tested), hands those numbers to Claude as a structured `Finding` , and Claude **narrates only** - it writes the exec Decision Brief and one recommended next test. The anti-hallucination doctrine here is different from M0-M6: there is no dbt compile or BigQuery dry-run to catch a wrong number, because the LLM is producing prose, not SQL. The single defense is **faithfulness**: every figure in the brief must trace verbatim to a tool output, and the prompt must forbid the model inventing, rounding, or recomputing any number. You build/test the deterministic layers first (they cost no LLM tokens), then add the brief on top.

Context to load

- `/clear` first (D8). Open a fresh session for M7 - no bleed from the dbt/MCP milestones.
- Auto-loaded: `CLAUDE.md` (repo root) - pins G1-G5, the canonical funnel, `revenue_at_risk = (conv_t0-conv_t1)*sessions_t1*aov` , and the faithfulness rule "Numbers in the brief are tool outputs verbatim" (G2).
- Source of truth to paste/attach: `@docs/architecture/AGENT_ARCHITECTURE.md` **sec 6.7 (Narrator)** - specifically the line "*every numeric claim must trace to a tool-output hash; no number the Narrator invented*" and the explicit note "*at M7 the Narrator only calls `render_brief` *"; **sec 4.3** for the `Finding` JSON shape that becomes the brief's input; **sec 13 step 2** for the L1 exit gate (one anomaly -> brief in <5 min, 0 hallucinated columns).
- `@docs/planning/LEAN_SCOPE.md` lines 76-80 - the v1 "one grounded LLM brief" definition and the stretch self-critique pass.
- `@docs/architecture/MCP_ARCHITECTURE.md` sec on `report.render_brief` (returns `{brief_md, brief_html}` , raises `EmptyFindings` on `[]`) - the only LLM-facing tool at M7.

Files to provide

- `@helios/diagnose.py` (from the MVP, Wk1) - the deterministic pipeline that finds the largest WoW movement in `session_conversion_rate` , decomposes it by `device_category` / `channel_group` , runs the two-proportion significance test, and computes the dollar figure. This produces the `Finding` ; M7 only adds a narration step after it.
- `@helios/stats/decompose.py` and the significance function - so Claude sees the exact keys it will receive (`mix_effect` , `rate_effect` , `interaction` , `dominant` , `p_value` , `ci_low` , `ci_high` , `revenue_at_risk_usd` , `basis`). It must narrate these, not regenerate them.
- `@models/semantic/semantic_layer.yaml` - the ONLY source of metric/dimension names the brief may use (D7).
- `@helios/mcp/<your-one-server>.py` (semantic-mcp OR stats-mcp from M6b) and `@mcp_servers.yaml` - the governed tool the deterministic layer calls.
- A real recorded `Finding` JSON (one run's output) as the test fixture, so the brief test needs no live LLM/BigQuery.

Prompts to use

Build the deterministic harness first (no LLM tokens spent), test-first:

```
[ROLE] You are implementing Helios milestone M7 (LEAN, Wk2-v1), file helios/brief.py.
[SOURCE OF TRUTH] Copy/adapt the Narrator contract in @docs/architecture/AGENT_ARCHITECTURE.md sec 6.7 and the Finding shape in sec 4.3. Follow CLAUDE.mc
[UPSTREAM] The Finding comes from @helios/diagnose.py (already built/tested). Reference only these keys for numbers: mix_effect, rate_effect, interactior
[TASK] Write helios/brief.py: a function build_brief(finding: dict) → str that calls the LLM exactly ONCE to compose a one-page exec Decision Brief (Sit
[CONSTRAINTS] The LLM may NOT invent, recompute, round, or restate any number not present in the Finding. Every figure in the brief MUST appear verbatim
[TEST-FIRST] Write helios/tests/test_brief_faithfulness.py FIRST: load the recorded Finding fixture, build the brief, then assert (a) every numeric toker
[OUTPUT] Cite the doc section (AGENT_ARCHITECTURE 6.7 / 4.3) used for the brief structure and the faithfulness rule.
```

The single grounded LLM-call prompt embedded inside `build_brief` (this is the product's runtime prompt - keep it terse and number-locked):

```
You are the Helios Narrator. Compose a one-page executive Decision Brief from the Finding JSON below.

HARD RULES:
- Every number you write MUST be copied verbatim from the Finding. Do NOT compute, re-derive, round, or estimate any figure. If a number is not in the Fi
- You did not run any analysis. The decomposition, significance test, and dollar figure are already computed and authoritative - narrate them, never secc
- Use only the metric and dimension names exactly as they appear in the Finding.
- State the mix-vs-rate verdict using finding.dominant: if "rate", the change is real in-segment behavior; if "mix", it is a composition shift (Simpson's
- One recommended next test only, as a single sentence; do not design or size an experiment.

Sections: Situation | What moved (metric, slice, t0→t1, delta) | Mix vs rate verdict | Revenue at risk (with the basis string) | Significance (p, CI) |

FINDING:
<finding_json>
```

(Stretch, if a day is free - the self-critique nod from LEAN_SCOPE) - a SECOND, cheap LLM call that audits the first against the numbers:

```
[ROLE] M7 stretch: a single self-critique pass in helios/brief.py (build_brief(finding, critique=True)).
[TASK] After composing the brief, make ONE more LLM call that receives the brief AND the Finding JSON and returns a JSON {faithful: bool, offending_spans
[CONSTRAINTS] The critic call may ONLY compare strings to the Finding; it does not fetch data or compute. This is a deterministic guardrail, not the M9 C
[TEST-FIRST] Add a test that feeds a deliberately corrupted brief (a hand-edited dollar figure) and asserts BriefFaithfulnessError is raised. Run pytest
[OUTPUT] Cite LEAN_SCOPE lines 79-80 (the self-critique stretch) and note this is NOT M9's Critic.
```

Preventing hallucinations (specialized - M7 has no compiler to catch a bad number)

- **The numbers come from tools, the LLM only narrates.** Build and test `helios/diagnose.py` + `decompose.py` + the significance test (deterministic, golden-value tested at M5) BEFORE writing `brief.py`. The LLM never sees raw rows and never computes - it receives a finished `Finding` (D2/D3 applied to a JSON payload instead of upstream SQL).
- **Forbid invention explicitly (G2 specialized).** The runtime prompt's HARD RULES forbid computing, re-deriving, rounding, or estimating any figure - the brief may only echo what is in the `Finding`. This is the M7 form of D5: "if a number is not in the attached `Finding`, OMIT it - do not guess."
- **Canonical names only (D7/G5).** The brief may name only metrics/dimensions present in `semantic_layer.yaml`; the test greps the brief for any metric-like token not in the registry and fails on a hit. No raw SQL is in scope at all (D2): the brief step calls zero query tools.
- **Let the test be the machine check (D6 analog).** dbt parse can't help here, so the substitute is `test_brief_faithfulness.py`: every numeric token in the brief must be a substring of the serialized `Finding` numbers. That is the automated faithfulness gate that replaces dry_run for the prose layer.
- **Don't let the LLM drift into "diagnosing why."** CLAUDE.md and LEAN_SCOPE forbid causal claims - the prompt says the model narrates *where* and *how much*, never *why*. A brief sentence asserting a cause (a deploy, a price change) is a hallucination of fact and must be cut.

Reviewing generated code (specialize R1-R6)

- **R-faithfulness (the M7-specific gate, replaces R6 dry_run):** for the rendered brief, every figure must trace to a `Finding` field. Diff manually: take each dollar amount, percentage, p-value, and CI bound in the brief and find it verbatim in the input `Finding`. Zero orphan numbers. The automated `test_brief_faithfulness.py` must be green AND you read the brief once yourself.
- **R0 SQL count:** confirm `brief.py` issues **0** queries and contains **0** SQL strings - the brief step only calls the LLM and (optionally) `report.render_brief`. `grep` the diff for `SELECT` / `build_query` inside `brief.py`: there should be none (all SQL lives upstream in the tested deterministic layer).
- **R1 cite:** the generated `brief.py` docstring cites AGENT_ARCHITECTURE 6.7 (Narrator faithfulness) and 4.3 (Finding shape); the runtime prompt is the M7 `render_brief`-only variant, not the full Narrator.
- **R2 import:** `python -c "import helios.brief"` clean; the function signature is `build_brief(finding: dict) → str`.
- **R3 ran the test:** Claude shows `pytest helios/tests/test_brief_faithfulness.py -q` passing, including the corrupted-brief case raising the error (if stretch built).

- **R4 canonical names:** every metric/dimension in the brief is in `semantic_layer.yaml` (grep). The `dominant` verdict in the prose matches `finding.dominant`.
- **R5 conventions + scope:** ONE LLM call per brief (two with self-critique) - not a loop, not an agent fleet. No FSM, no `save_diagnosis`, no memory. `revenue_at_risk` basis string is echoed, not recomputed. Verify wall-clock: the deterministic run + one brief completes in <5 min (L1 gate, AGENT_ARCHITECTURE sec 13).
- **Budget note:** run the brief sparingly during dev - test `brief.py` against the recorded fixture (no LLM), and reserve live LLM calls for final v1 validation and the M10 eval. Do not loop the brief while iterating on prompt wording; iterate against the fixture and the faithfulness test.

M8 - Memory Store [v2 - DEFERRED]

Deferred. Per LEAN_SCOPE (lines 40, 47, 90), memory / vector store / suppression / `action_tracking` is **cut** for the 2-week v1: it is over-engineering for a batch job on 3 months of frozen data, and the seasonality calendar only earns its keep once it *demonstrably reduces false positives*. The M7 brief does not need it. The notes below are how you *would* drive Claude Code to build it later - do not spend Pro budget or the 2 weeks on it now.

Context to load - `/clear`; rely on auto-loaded `CLAUDE.md`; attach `@docs/architecture/HELIOS_PROJECT_BIBLE.md` **sec 22** (the 8 `helios_memory` tables, the seasonality/launch calendars, the `exp(-age/60d)` recall decay) and `@docs/architecture/MCP_ARCHITECTURE.md` sec 6.5 + sec on `save_diagnosis` / `recall_prior` (the only memory write path; idempotent on `finding_id`).

Files to provide - `@docs/architecture/HELIOS_PROJECT_BIBLE.md` sec 22 as the verbatim DDL source (the `CREATE TABLE IF NOT EXISTS` for `diagnosis_history`, `suppression_list`, `seasonality_calendar`, `launch_calendar`, `action_tracking`, `run_state`, `audit_log`, `glossary`); `@models/semantic/semantic_layer.yaml` for the canonical names the glossary maps to; the existing `@helios/mcp/report.py` to extend.

Prompts to use (sketch) - one focused session: "*[ROLE] M8 (v2), file `sql/helios_memory_ddl.sql`. [SOURCE OF TRUTH] copy the 8 `CREATE TABLE` statements verbatim from `@HELIOS_PROJECT_BIBLE.md` sec 22, including PARTITION/CLUSTER. [TASK] the DDL + a `seeds/memory/seasonality_calendar.csv` seeded with Black Friday 2020, the December peak, and the January trough. [CONSTRAINTS] use ONLY sec 22 column names; do not invent fields. [TEST-FIRST] write a `save_diagnosis → recall_prior` round-trip test against a fixture, then implement the `report-mcp` memory tools to pass it.*" A second prompt adds `save_diagnosis` / `recall_prior` to `report.py` keyed idempotently on `finding_id`.

Preventing hallucinations - D2/D3 with Bible sec 22 as the verbatim source; D5 forbids inventing memory columns; the round-trip test is the machine check (D6) - a saved finding must be retrievable by `(metric, segment)` and re-saving the same `finding_id` upserts (not duplicates). Seed the calendar from real dataset events only.

Reviewing generated code - R1 cites sec 22; R2 the DDL runs in BigQuery (`--location=US` to match the marts); R3 the round-trip test passes; R5 `report-mcp` is the ONLY memory writer (no agent holds a raw insert path), recall decays with `exp(-age/60)` and never hard-deletes (audit requirement).

Why deferred (be explicit): building 8 tables + a vector store + idempotent memory tools is several days and adds recurring LLM/embedding cost - it would blow the 2-week budget for a feature the frozen 3-month dataset cannot validate. Defer until v1 ships and you can show it cuts false positives.

M9 - Full 7-Agent Loop [v2 - DEFERRED]

Deferred. Per LEAN_SCOPE (lines 37, 90, 98) the 7-agent FSM is the single biggest cut: it is *apparatus disproportionate* to the value, and Claude Pro **cannot fund a fleet** of 7 agents x many BigQuery round-trips per run (LEAN_SCOPE line 21). The value - the decomposition and the grounding - is already delivered by M7's one grounded brief. The notes below are how you *would* drive Claude Code to build the full loop later; doing it inside the 2 weeks would exhaust both the calendar and the Pro budget.

Context to load - `/clear`; auto-loaded `CLAUDE.md`; attach `@docs/architecture/AGENT_ARCHITECTURE.md` **sec 4-10** (the `AgentSpec`, the tool-call wrapper, the `Finding` envelope, the FSM + sec 5.1 transition table, the per-agent specs 6.1-6.7, the G1-G5 double enforcement in sec 7, and sec 9 tunables) - plus `@docs/architecture/MCP_ARCHITECTURE.md` sec 10 (the authoritative per-agent allow-lists) and M8's memory tools (needed for the Critic's seasonality priors and the Narrator's `save_diagnosis`).

Files to provide - the M7 `helios/agents/framework.py` + `runner.py` to extend; `@docs/architecture/AGENT_ARCHITECTURE.md` sec 6.x per agent; `@models/semantic/semantic_layer.yaml` (the only metrics agents may name); recorded MCP fixtures for node unit tests (no live BigQuery).

Prompts to use (sketch) - this is where **subagents earn their keep** (the one place LEAN_SCOPE's "reserve subagents" rule relaxes): the five remaining agent files (`decompose.py`, `diagnose.py`, `critic.py`, `prescribe.py`, `orchestrator.py`) are largely independent, each a `*_SPEC` +

system prompt + node logic copied from its sec 6.x. Drive one code-writing subagent per agent file in parallel, each handed only its own sec 6.x + its sec-10 allow-list, e.g.: "*[ROLE] M9 (v2), file `helios/agents/critic.py`. [SOURCE OF TRUTH] copy the `CRITIC_SPEC` + 4-axis refutation battery from `@AGENT_ARCHITECTURE.md` sec 6.5; tools = exactly its `@MCP_ARCHITECTURE.md` sec 10 row. [TASK] the Critic node + its refutation logic. [CONSTRAINTS] the Critic's job is to REFUTE, default to skepticism, PASS only if all four refutations fail; no raw SQL (G1), no prose stats (G2). [TEST-FIRST] write `test_critic.py` with a mislabeled pure-mix-shift fixture asserting DROP/DOWNGRADE and a clean fixture asserting PASS, then implement.*" A final non-parallel prompt wires the full `PLAN→MONITOR→DECOMPOSE→DIAGNOSE→[CRITIC]→PRESCRIBE→NARRATE` FSM from sec 5.1. Use the **Critic as a self-check pass** - it is the production analog of M7's stretch self-critique, but with the full mix-shift / sample / seasonality / data-quality battery.

Preventing hallucinations - D2/D3 with sec 6.x as the verbatim source per agent; D5 forbids inventing tools or metrics; the structural defenses do the heavy lifting (sec 7): the framework wrapper `assert tool in agent.allowed_tools` makes out-of-scope tools impossible and `assert dry_run_seen(sql)` enforces G3 - so "the agent wrote its own SQL" is structurally impossible. The machine checks (D6): node unit tests against recorded fixtures, FSM transition tests, and the `audit_log` grounding scan (every `sql_text` originated from `semantic.build_query`; every brief number maps to a tool-output hash).

Reviewing generated code - R1 each agent cites its sec 6.x; R2 nodes import and validate their `Finding` against `output_schema`; R3 `test_critic.py` / `test_fsm_full.py` pass; R4 only `semantic_layer.yaml` names appear; R5 each agent's allowed_tools matches sec 10 exactly (e.g. Narrator cannot call `run_query`, Prescribe cannot touch `warehouse`) and Diagnose drills *rate* before *mix*; R6 leaf promotion requires `reconcile` <=0.5% drift.

Why deferred (be explicit): seven agents x Opus/Sonnet calls x multiple tool round-trips per run is an *API-fleet* cost profile that Claude Pro's usage limits cannot sustain - and the eval (M10) shows the lean one-brief path already beats the naive baseline. Build the full loop only as a funded v2, after M7/M10 prove the spine. The honest interview line is exactly this scoping decision (LEAN_SCOPE line 126): the full design was a 7-agent autonomous system; with 2 weeks and one dev you shipped the governed spine + one grounded LLM step + an honest eval, because the value was the decomposition and the grounding, not the fleet.

M10 - Eval Harness (injector + scorer + 6-10 scenarios + naive baseline) [Wk2-v1 LEAN]

This is the v1 honest-eval deliverable and the trust centerpiece of the whole project. LEAN means: build `injector.py` + `scorer.py`, **reuse a 6-10 scenario subset** of the existing `eval/scenarios/scenarios.yaml` (do NOT regenerate the 50, do NOT build CI), and the naive "largest-absolute-delta" baseline. The eval runs **locally** (`python -m helios.eval`). The anti-overclaim move (the thing that wins interview rounds): make Claude state plainly that this proves *controlled attribution accuracy vs a baseline* and NOT real-world causal accuracy.

Context to load

- Auto-loaded: `CLAUDE.md` (canonical metrics, the core decomposition identity, the 85%-vs-45% targets, grounding rules G1-G5).
- The source of truth: HELIOS_PROJECT_BIBLE.md **sec 20** (Evaluation Framework) - specifically 20.2 injection mechanism (the two perturbation primitives), 20.4 metrics (top-1/top-3 segment accuracy, decomposition MAPE, dollar-at-risk error), 20.5 the naive baseline definition, 20.7 scoring details (normalized segment-key matching).
- `docs/planning/LEAN_SCOPE.md` lines ~43, ~78, ~125 - the LEAN cut (6-10 scenarios, no CI) and the exact honesty sentence the eval must print.
- The format reference + reusable specs: `eval/scenarios/scenarios.yaml` (the 50-scenario master list) and the per-bucket files `eval/scenarios/01_single_segment_rate.yaml` ... `07_data_quality.yaml`. You are picking ~8 of these, not authoring new ones.
- The thing being graded: the M7 grounded LLM brief output (the JSON diagnosis your minimal loop emits) and `stats-mcp.decompose_change` (M6b) - the scorer reads their structured output.
- /clear before starting (D8). M10 is its own focused session.

Files to provide

- `@HELIOS_PROJECT_BIBLE.md` (sec 20 only - paste/attach the section, not the whole Bible).
- `@eval/scenarios/scenarios.yaml` (the master format reference) and the picked subset files.
- `@models/semantic/semantic_layer.yaml` and the GA4 schema - the scorer's `hallucination` check needs the canonical column/metric registry (D7).
- The M7 diagnosis output schema (the dict your loop returns: predicted `root_cause_segment`, predicted `dominant_effect`, the decomposition `{mix, rate, interaction}`, `revenue_at_risk_usd`).
- `@helios/mcp/stats.py` (the `decompose_change` contract the harness drives) and `@helios/mcp/warehouse.py` (`run_query` / `dry_run` to materialize `helios_eval.fct_daily_funnel_perturbed`).

Prompts to use

Pick the subset first (cheap, plan-mode, no codegen):

```
[ROLE] You are setting up the Helios LEAN eval (milestone M10). Plan only - no code yet.
[SOURCE OF TRUTH] Bible sec 20.3 (coverage buckets) + the attached eval/scenarios/scenarios.yaml.
[TASK] From the existing 50 scenarios, SELECT exactly 8 to form the LEAN benchmark, covering:
  2 single_segment_rate (e.g. S001, S002), 2 single_segment_mix (e.g. S011, S014 the Simpson's adversarial),
  1 multi_segment_rate (e.g. S021), 1 multi_segment_mixed (e.g. S027), 2 no_anomaly_control (e.g. S039, S040).
[CONSTRAINTS] Do NOT write new scenarios and do NOT modify the YAML. Reuse the exact specs as-is.
  Justify each pick against the bucket it covers and why it discriminates Helios from the naive baseline
  (the mix + adversarial picks are the ones the baseline must fail).
[OUTPUT] A table: scenario_id, bucket, why-picked. Cite the bucket rows in Bible sec 20.3.
```

Then the injector (TDD, one file):

```
[ROLE] You are implementing Helios milestone M10, file helios/eval/injector.py.
[SOURCE OF TRUTH] Copy/adapt the injection mechanism in Bible sec 20.2 (attached). Follow CLAUDE.md.
[UPSTREAM] Read scenario specs from eval/scenarios/*.yaml (attached format). Materialize via
  warehouse-mcp.run_query/dry_run (helios/mcp/warehouse.py). Operate ONLY on helios_eval - never the public source.
[TASK] Implement inject(spec) with the TWO primitives from sec 20.2 and NOTHING else:
  - rate perturbation: recompute numerator = round(sessions * base_rate * rate_multiplier), hold sessions fixed → ground_truth dominant_effect = rate.
  - volume/mix perturbation: multiply segment sessions by volume_multiplier, hold per-step rates fixed, renormalize totals → dominant_effect = mix.
  Write helios_eval.fct_daily_funnel_perturbed + a labels record (scenario_id, anomaly_type, affected segment, the analytic mix/rate/interaction split, t
[CONSTRAINTS] Use ONLY canonical column names (sessions, view_item_sessions, add_to_cart_sessions,
  begin_checkout_sessions, purchasing_sessions, revenue, transactions) and the fct_daily_funnel grain. Do NOT
  invent columns. If a name is not in semantic_layer.yaml / the GA4 schema, STOP and ASK.
[TEST-FIRST] Write tests/eval/test_injector.py FIRST: for a rate scenario assert sessions are
  byte-identical pre/post and only the numerator changed; for a mix scenario assert each segment's per-step
  rate is unchanged and total sessions is conserved within tolerance. Run `pytest tests/eval/test_injector.py`
  and show output.
[OUTPUT] Cite the sec 20.2 sub-bullet used for each primitive.
```

Then the scorer + naive baseline (TDD, one file each - the honesty lever lives here):

```
[ROLE] You are implementing Helios milestone M10, files helios/eval/scorer.py and helios/eval/baseline.py.
[SOURCE OF TRUTH] Bible sec 20.4 (metrics) + 20.5 (naive baseline) + 20.7 (scoring details). Follow CLAUDE.md.
[UPSTREAM] Read predicted diagnosis from the M7 loop output dict and ground-truth from helios_eval.labels.
[TASK]
  baseline.py: implement the naive "largest-absolute-segment-delta" analyst exactly per sec 20.5 -
    for the anomalous metric, delta_i = value_at(t1) - value_at(t0) per segment, rank by |delta|, declare the
    top segment the root cause. NO mix-vs-rate decomposition (that is the point - it gets fooled by mix).
  scorer.py: compute, in DETERMINISTIC PYTHON ONLY (NEVER an LLM judge), per sec 20.4:
    top-1 and top-3 root-cause segment accuracy (normalized sorted dimension=value key match, sec 20.7),
    decomposition MAPE on {mix, rate, interaction} (controls excluded), dollar-at-risk abs % error,
    and an AST-based hallucination check against semantic_layer.yaml + GA4 schema.
[CONSTRAINTS] The scorer must be pure Python/scipy - do NOT call Claude to grade. Use only canonical names.
  HONESTY: the final report MUST print a one-line caveat verbatim: "This benchmark measures controlled
  attribution accuracy (where the number moved) against a naive baseline. It does NOT prove real-world causal
  accuracy - the cause (a deploy, a price change) lives outside this frozen dataset."
[TEST-FIRST] Write tests/eval/test_scorer.py FIRST with a hand-worked golden case (a known mix scenario where
  the baseline picks the wrong high-volume segment and the decompose-based prediction picks the right one) and
  assert: baseline top-1 = miss, Helios top-1 = hit, MAPE within tolerance. Run pytest and show output.
[OUTPUT] Cite sec 20.4/20.5/20.7 per function.
```

Preventing hallucinations

- D2/D3 specialized: the scenarios already exist with exact ground-truth labels - hand Claude `scenarios.yaml` and forbid it from writing new specs. The number-one drift here is Claude "improving" the scenarios; the prompt says reuse as-is.
- D6 (let the machine catch it): the injector is graded by its own invariants - the `test_injector.py` rate-test asserts `sessions` are byte-identical pre/post (a rate perturbation that touches volume is a bug); the mix-test asserts per-segment rates are unchanged. These are the silent-failure tripwires from CLAUDE.md sec 8.
- D7 hard rule: the scorer's hallucination check is itself the column-name guard - point it at `semantic_layer.yaml` + the GA4 schema; any emitted column not in those is a fail. Keep both attached.
- The honesty/anti-overclaim guard is a *generation* constraint, not just review: the prompt forces the verbatim caveat line and forbids claiming "causal" or ">=85% real-world accuracy." If Claude writes "Helios proves it diagnoses *why*," reject - the doc (LEAN_SCOPE line ~125) only permits "*where* it moved and *how much it's worth*."
- D5: if Claude reaches for a scoring metric not in sec 20.4 (e.g. an "LLM faithfulness judge"), STOP - faithfulness/Critic-as-judge is [v2], explicitly cut from LEAN. The LEAN scorer is four deterministic numbers, period.

Reviewing generated code

- R1: each scorer function cites sec 20.4/20.5/20.7; the injector cites the two 20.2 primitives.

- R3 (run it): `python -m helios.eval` end-to-end on the 8 scenarios. **The two sanity checks that matter most** (this is the review, not a formality):
    1. The baseline beats *nothing* - it must score clearly above 0 on the pure single-segment-rate scenarios (S001/S002) and clearly *fail* the mix/adversarial ones (S011/S014). If the baseline scores ~0 everywhere your baseline is broken (too weak a strawman); if it scores high on mix, it isn't the largest-delta rule.
    2. Helios beats the baseline - top-1 materially higher than the baseline, driven by the mix scenarios the decomposition handles. If they tie, the eval proves nothing - investigate before reporting a number.
- R4/R6: grep the scorer + injector emitted SQL/columns against `semantic_layer.yaml`; hallucination count must be 0. Confirm the scorer is **not** graded by an LLM (read `scorer.py` - it must be scipy/Python, no `anthropic` / `messages.create` import). This is a hard reject if violated.
- R5: read the report output - it must print the honesty caveat verbatim and must NOT print an unqualified "85% causal accuracy." Report the *actual measured* numbers on your 8 scenarios, not the Bible's aspirational 0.882.
- Budget note: do NOT re-run the M7 LLM brief on every scenario during dev (that burns Pro usage). Develop injector + scorer + baseline against *recorded* M7 outputs / fixtures; run the live LLM loop through the 8 scenarios ONCE to validate v1.
- Per-scenario debugging: have the harness emit a per-scenario JSON (predicted vs label, every sub-score) per Bible sec 20.6 so a miss is inspectable. When you find a miss, read that JSON before re-prompting Claude - usually it is a segment-key normalization bug (sec 20.7: sorted dimension=value match), not a logic error, and you fix the matcher, not the diagnosis.

Slash command to create here (reuse later). `/new-scenario` - scaffold one eval scenario (the full spec block + its expected ground-truth labels) from a one-line description, validated against the `scenarios.yaml` field set in CLAUDE.md. Keep it for when v2 reintroduces buckets (seasonality-decoy, data-quality) the LEAN 8 omitted; for v1 you mostly *select* existing scenarios rather than author new ones, so this command stays dormant until you genuinely need an uncovered bucket. Pair it with a quick re-run of the injector invariants test so a hand-written spec cannot silently violate the rate/mix primitives.

M11 - Autonomy & depth [v2-later]

Out of the 2-week / Claude-Pro scope, and partly theater on a frozen dataset. The red-team explicitly killed "always-on / scheduler / autonomous" (LEAN_SCOPE line ~39: "*Run it as a command. Why: theater on a frozen dataset.*"). A scheduler that re-diagnoses a static 2020-11-01->2021-01-31 export produces the same answer every run - the autonomy is real engineering but adds no signal on this data. Forecasting/cohort depth (Bible sec 23.3) is genuine v2, deferred for time, not because it is unsound.

How you would drive Claude Code later (sketch). /clear, then plan-mode against Bible sec 23.3 + the deferred-items table (sec 23.7). Build in three thin slices, each its own session: (1) a scheduler wrapper - Claude scaffolds a Cloud Scheduler/cron entry that just invokes the existing `python -m helios.run`; no new diagnosis logic. (2) Forecast wiring - `[ROLE] implement stats-mcp.forecast (prophet/pmdarima) per Bible sec 23.3; expected-vs-actual residual feeds Monitor.detect_anomaly`; TDD with a golden series so the forecast is deterministic-seeded math, not prose (G3). (3) `cohort_retention` / `rfm_segment` feeding the Diagnose hypothesis tree - but cap retention at the 30/60-day proxy the data supports (sec 23.7: the ~3-month window is too short for true LTV; have Claude assert and print that limit rather than fabricate a curve). The Critic refutation battery (mix-shift/sample/seasonality/data-quality) also lands here.

Anti-hallucination + honesty (the v2 carry-over). Same doctrine: source-of-truth section attached, only canonical names, every stat through `stats-mcp` seeded. The specific overclaim to police: a scheduler does not make Helios "autonomous" on frozen data, and forecast residuals on a closed historical export are a demo, not a live signal - Claude must label them as such. Review by running the forecast against a held-back window and checking residual sign/magnitude by hand; never accept a forecast Claude narrated without the tool output.

M12 - Productionization & frontier [v2-later]

Furthest out of scope; the frontier half is partly theater on this dataset by construction. Bible sec 23.4 (multi-tenant, real-time streaming, warehouse-agnostic) and 23.5 (true causal inference, closed-loop experimentation) are weeks of platform + causal-ML work for one dev. More importantly, the red-team's central honesty point (LEAN_SCOPE line ~125): the public data has **no experiment assignment** (sec 23.7), so true causal inference cannot be *validated* here - the frontier exit criterion ("a causal estimate validated against a held-out randomized experiment") is unmeetable on this sample. Multi-tenant and streaming add no signal to a single frozen export. These earn their place on the *roadmap narrative*, not the build.

How you would drive Claude Code later (sketch). /clear; plan against sec 23.4-23.5 + the capability-maturity table (sec 23.6). Slices, each isolated: (1) warehouse-agnostic adapters behind `warehouse-mcp` (Snowflake/Databricks/DuckDB) - `[ROLE] implement a DuckDB adapter behind the existing warehouse-mcp contract`; review with the exit criterion "two warehouses pass identical `reconcile` tests" (sec 23.4) - the semantic layer stays the only SQL author regardless of dialect. (2) Tenant isolation - per-tenant semantic layer + byte budgets; (3) causal methods (diff-in-diff / synthetic control / double-ML) *only* where data supports it, replacing correlational decomposition - and Claude must gate each on a data-sufficiency check, refusing to emit a causal estimate the sample cannot support.

Anti-hallucination + honesty. This is where overclaiming is most dangerous and most tempting. The hard rule for Claude: never let the Narrator upgrade "decomposition attributes *where* the change is" into "Helios proves *why*" - the cause lives outside the data (LEAN_SCOPE line ~125, the maturity move). Any causal language must carry the confound caveat the Critic attaches. Review by demanding the data-sufficiency assertion in code (a causal estimate without an assignment mechanism is rejected) and by reading every frontier brief for unqualified causal claims - on a frozen observational sample those are the exact statements that "get you destroyed in an interview" (LEAN_SCOPE line ~14). For v2, the deliverable is honest roadmap prose plus the scoping rationale, not running causal code against data that cannot validate it.

Quick Reference â€" per-milestone cheat sheet

M	Name	Tag	Source of truth to attach	The one test that proves it	Top hallucination risk
M0	Foundation/toolchain	Wk1	DBT_GUIDE Â§1, Bible Â§16â€"17	`dbt debug` green	wrong IAM role / dataset location not US
M1	Sources, macros, seed	Wk1	DBT_GUIDE Â§2â€"3, Â§6	macros compile; `dbt seed`	inventing `event_params` keys / a 2nd channel macro
M2	Staging	Wk1	DBT_GUIDE Â§3	`dbt build --select staging`	hallucinated raw GA4 columns
M3	**Sessionization + funnel (KEYSTONE)**	**Wk1**	DBT_GUIDE Â§4, DATA_MODEL Â§5	**monotonicity + `session_key` uniqueness (golden, first)**	`did_*` vs `reached_*` `traffic_source` as session source; wrong `session_key`
M4	Marts	Wk1	DBT_GUIDE Â§5, DATA_MODEL Â§3â€"8	`revenue_reconciles` to the cent; channels = 10	hallucinated columns; no `transaction_id` dedup
M5	Semantic layer live	Wk1	`semantic_layer.yaml` (exists), METRIC_GOVERNANCE_GUIDE	`validate_semantic.py` â†' 0 dangling refs	inventing metrics; wrong registry filename
M6	semantic-mcp + warehouse-mcp	Wk2	MCP_ARCHITECTURE Â§6.1â€"6.2, Â§9	`build_queryâ†'dry_runâ†'run_queryâ†'reconcile` round-trip; budget gate	the LLM hand-writing SQL
M6b	stats / experiment / report MCP	Wk2Â[1]	MCP_ARCHITECTURE Â§6.3â€"6.5, Â§9	`decompose_change` golden test	computing stats in prose
M7	Minimal loop (LEAN = 1 brief)	Wk2	AGENT_ARCHITECTURE Â§4, Â§6, Â§13	every brief figure traces to a tool output	the LLM inventing/recomputing numbers
M8	Memory	v2	Bible Â§22	`save_diagnosis` â†' `recall_prior`	â€"
M9	Full 7-agent loop	v2	AGENT_ARCHITECTURE Â§5â€"Â§10	FSM routing	â€"
M10	Eval harness (LEAN = 6â€"10, local)	Wk2	Bible Â§20, `scenarios.yaml` (exists)	Helios beats the naive baseline	over-claiming *causal* accuracy; an LLM-graded scorer
M11	Autonomy & depth	v2	Bible Â§23.3	scheduled run	autonomy theater on a frozen dataset
M12	Productionization/frontier	v2	Bible Â§23.4â€"5	â€"	â€"

Â[1] M6b in v1 = build only the **one** server you chose (semantic *or* stats); the rest are v2.

The golden rules (tape these to your monitor)

1. **`CLAUDE.md` at the root is your hallucination firewall** â€" never code without it loaded.
2. **No file without its source-of-truth doc section + the real upstream files attached.**
3. **Plan mode before any multi-file milestone.**
4. **Test-first** â€" and make Claude *run* the test and show you the output (don't trust, verify).

5. **The machine catches hallucinations** â€” `compile` + `dry_run` + `test` before you accept anything.
6. `/clear` **between milestones.**
7. **Numbers come from tools, not the LLM** (M7+) â€” the model narrates; `stats-mcp` computes.
8. **Spend LLM budget only on the brief** â€” build the deterministic layers with Claude Code.
9. **If Claude invents a name, that's a defect** â€” reject it, attach the registry, regenerate.

The honesty discipline (it's also a hallucination guard)

When Claude writes the M7 brief or the M10 eval writeup, make it state plainly what is *measured* vs *assumed*: the decomposition shows *where* a metric moved (attribution), not *why* (causation); the eval proves controlled-attribution accuracy vs a baseline, not real-world causal accuracy. Forcing this honesty prevents the model from generating confident causal stories the data can't support â€” the exact failure the red-team flagged.